

Prashanth Terupelli Portfolio - Complete Code Documentation

Overview

This is the complete source code documentation for the Prashanth Terupelli Portfolio. It includes the frontend React application, the backend Express server, tRPC routers, database schemas, and all configuration files.

File: ./client/index.html

```
<!doctype html>
<html lang="en">

  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0, maximum-scale=1" />
    <title>Prashanth Terupelli | Senior Product Analyst Portfolio</title>
    <!-- THIS IS THE START OF A COMMENT BLOCK, BLOCK TO BE DELETED: Google
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800;900" rel="stylesheet" />
    THIS IS THE END OF A COMMENT BLOCK, BLOCK TO BE DELETED -->
  </head>

  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
    <script
      defer
      src="%VITE_ANALYTICS_ENDPOINT%/umami"
      data-website-id="%VITE_ANALYTICS_WEBSITE_ID%"></script>
  </body>

</html>
```

File: ./client/src/App.tsx

```
import { Toaster } from "@components/ui/sonner";
import { TooltipProvider } from "@components/ui/tooltip";
import NotFound from "@pages/NotFound";
import { Route, Switch } from "wouter";
import ErrorBoundary from "../components/ErrorBoundary";
import { ThemeProvider } from "../contexts/ThemeContext";
import Home from "../pages/Home";

function Router() {
  // make sure to consider if you need authentication for certain routes
  return (
    <Switch>
      <Route path="/" component={Home} />
      <Route path="/404" component={NotFound} />
      {/* Final fallback route */}
      <Route component={NotFound} />
    </Switch>
  );
}

// NOTE: About Theme
// - First choose a default theme according to your design style (dark or
//   to keep consistent foreground/background color across components
// - If you want to make theme switchable, pass `switchable` ThemeProvider

function App() {
  return (
    <ErrorBoundary>
      <ThemeProvider
        defaultTheme="dark"
        // switchable
      >
        <TooltipProvider>
          <Toaster />
          <Router />
        </TooltipProvider>
      </ThemeProvider>
    </ErrorBoundary>
  );
}
```

```
export default App;
```

File: ./client/src/components/AIChatBox.tsx

```
import { Button } from "@components/ui/button";
import { Textarea } from "@components/ui/textarea";
import { ScrollArea } from "@components/ui/scroll-area";
import { cn } from "@lib/utils";
import { Loader2, Send, User, Sparkles } from "lucide-react";
import { useState, useEffect, useRef } from "react";
import { Streamdown } from "streamdown";

/**
 * Message type matching server-side LLM Message interface
 */
export type Message = {
  role: "system" | "user" | "assistant";
  content: string;
};

export type AIChatBoxProps = {
  /**
   * Messages array to display in the chat.
   * Should match the format used by invokeLLM on the server.
   */
  messages: Message[];

  /**
   * Callback when user sends a message.
   * Typically you'll call a tRPC mutation here to invoke the LLM.
   */
  onSendMessage: (content: string) => void;

  /**
   * Whether the AI is currently generating a response
   */
  isLoading?: boolean;

  /**
   * Placeholder text for the input field
   */
  placeholder?: string;

  /**
   * Custom className for the container
   */
}
```

```

    */
    className?: string;

    /**
     * Height of the chat box (default: 600px)
     */
    height?: string | number;

    /**
     * Empty state message to display when no messages
     */
    emptyStateMessage?: string;

    /**
     * Suggested prompts to display in empty state
     * Click to send directly
     */
    suggestedPrompts?: string[];
};

/**
 * A ready-to-use AI chat box component that integrates with the LLM system
 *
 * Features:
 * - Matches server-side Message interface for seamless integration
 * - Markdown rendering with Streamdown
 * - Auto-scrolls to latest message
 * - Loading states
 * - Uses global theme colors from index.css
 *
 * @example
 * ```tsx
 * const ChatPage = () => {
 *   const [messages, setMessages] = useState<Message[]>([
 *     { role: "system", content: "You are a helpful assistant." }
 *   ]);
 *
 *   const chatMutation = trpc.ai.chat.useMutation({
 *     onSuccess: (response) => {
 *       // Assuming your tRPC endpoint returns the AI response as a string
 *       setMessages(prev => [...prev, {
 *         role: "assistant",
 *         content: response
 *       }]);
 *     }
 *   });
 *
 *   return (
 *     <ChatBox
 *       messages={messages}
 *       chatMutation={chatMutation}
 *     />
 *   );
 * };
 *
 * export default ChatPage;
 */

```

```

*     },
*     onError: (error) => {
*         console.error("Chat error:", error);
*         // Optionally show error message to user
*     }
* });
*
* const handleSend = (content: string) => {
*     const newMessages = [...messages, { role: "user", content }];
*     setMessages(newMessages);
*     chatMutation.mutate({ messages: newMessages });
* };
*
* return (
*     <AIChatBox
*         messages={messages}
*         onSendMessage={handleSend}
*         isLoading={chatMutation.isPending}
*         suggestedPrompts={[
*             "Explain quantum computing",
*             "Write a hello world in Python"
*         ]}
*     />
* );
* };
*
*/
export function AIChatBox({
    messages,
    onSendMessage,
    isLoading = false,
    placeholder = "Type your message...",
    className,
    height = "600px",
    emptyStateMessage = "Start a conversation with AI",
    suggestedPrompts,
}: AIChatBoxProps) {
    const [input, setInput] = useState("");
    const scrollAreaRef = useRef<HTMLDivElement>(null);
    const containerRef = useRef<HTMLDivElement>(null);
    const inputAreaRef = useRef<HTMLFormElement>(null);
    const textareaRef = useRef<HTMLTextAreaElement>(null);

    // Filter out system messages

```

```

const displayMessages = messages.filter((msg) => msg.role !== "system")

// Calculate min-height for last assistant message to push user message
const [minHeightForLastMessage, setMinHeightForLastMessage] = useState(0)

useEffect(() => {
  if (containerRef.current && inputAreaRef.current) {
    const containerHeight = containerRef.current.offsetHeight;
    const inputHeight = inputAreaRef.current.offsetHeight;
    const scrollAreaHeight = containerHeight - inputHeight;

    // Reserve space for:
    // - padding (p-4 = 32px top+bottom)
    // - user message: 40px (item height) + 16px (margin-top from space)
    // Note: margin-bottom is not counted because it naturally pushes the
    const userMessageReservedHeight = 56;
    const calculatedHeight = scrollAreaHeight - 32 - userMessageReservedHeight;

    setMinHeightForLastMessage(Math.max(0, calculatedHeight));
  }
}, []);

// Scroll to bottom helper function with smooth animation
const scrollToBottom = () => {
  const viewport = scrollAreaRef.current?.querySelector(
    '[data-radix-scroll-area-viewport]'
  ) as HTMLDivElement;

  if (viewport) {
    requestAnimationFrame(() => {
      viewport.scrollTo({
        top: viewport.scrollHeight,
        behavior: 'smooth'
      });
    });
  }
};

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  const trimmedInput = input.trim();
  if (!trimmedInput || isLoading) return;

  onSendMessage(trimmedInput);

```

```

    setInput("");

    // Scroll immediately after sending
    scrollToBottom();

    // Keep focus on input
    textareaRef.current?.focus();
  };

  const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
    if (e.key === "Enter" && !e.shiftKey) {
      e.preventDefault();
      handleSubmit(e);
    }
  };

  return (
    <div
      ref={containerRef}
      className={cn(
        "flex flex-col bg-card text-card-foreground rounded-lg border shadow
        className
      )}
      style={{ height }}
    >
      {/* Messages Area */}
      <div ref={scrollAreaRef} className="flex-1 overflow-hidden">
        {displayMessages.length === 0 ? (
          <div className="flex h-full flex-col p-4">
            <div className="flex flex-1 flex-col items-center justify-center">
              <div className="flex flex-col items-center gap-3">
                <Sparkles className="size-12 opacity-20" />
                <p className="text-sm">{emptyStateMessage}</p>
              </div>

              {suggestedPrompts && suggestedPrompts.length > 0 && (
                <div className="flex max-w-2xl flex-wrap justify-center gap-2">
                  {suggestedPrompts.map((prompt, index) => (
                    <button
                      key={index}
                      onClick={() => onSendMessage(prompt)}
                      disabled={isLoading}
                      className="rounded-lg border border-border bg-card p-2"
                    >

```

```

        {prompt}
      </button>
    )}}
  </div>
)}
</div>
</div>
) : (
  <ScrollArea className="h-full">
    <div className="flex flex-col space-y-4 p-4">
      {displayMessages.map((message, index) => {
        // Apply min-height to last message only if NOT loading (
        const isLastMessage = index === displayMessages.length - 1
        const shouldApplyMinHeight =
          isLastMessage && !isLoading && minHeightForLastMessage :

        return (
          <div
            key={index}
            className={cn(
              "flex gap-3",
              message.role === "user"
                ? "justify-end items-start"
                : "justify-start items-start"
            )}
            style={
              shouldApplyMinHeight
                ? { minHeight: `${minHeightForLastMessage}px` }
                : undefined
            }
          >
            {message.role === "assistant" && (
              <div className="size-8 shrink-0 mt-1 rounded-full b
                <Sparkles className="size-4 text-primary" />
              </div>
            )}

            <div
              className={cn(
                "max-w-[80%] rounded-lg px-4 py-2.5",
                message.role === "user"
                  ? "bg-primary text-primary-foreground"
                  : "bg-muted text-foreground"
              )}

```

```

>
    {message.role === "assistant" ? (
      <div className="prose prose-sm dark:prose-invert">
        <Streamdown>{message.content}</Streamdown>
      </div>
    ) : (
      <p className="whitespace-pre-wrap text-sm">
        {message.content}
      </p>
    )}
  </div>

  {message.role === "user" && (
    <div className="size-8 shrink-0 mt-1 rounded-full bg-primary">
      <User className="size-4 text-secondary-foreground">
        {message.content}
      </div>
    )}
  </div>
);
})}

{isLoading && (
  <div
    className="flex items-start gap-3"
    style={
      minHeightForLastMessage > 0
        ? { minHeight: `${minHeightForLastMessage}px` }
        : undefined
    }
  >
    <div className="size-8 shrink-0 mt-1 rounded-full bg-primary">
      <Sparkles className="size-4 text-primary" />
    </div>
    <div className="rounded-lg bg-muted px-4 py-2.5">
      <Loader2 className="size-4 animate-spin text-muted-foreground" />
    </div>
  </div>
  </div>
)}
</div>
</ScrollArea>
)}
</div>

{ /* Input Area */ }

```

```

<form
  ref={inputAreaRef}
  onSubmit={handleSubmit}
  className="flex gap-2 p-4 border-t bg-background/50 items-end"
>
  <Textarea
    ref={textareaRef}
    value={input}
    onChange={(e) => setInput(e.target.value)}
    onKeyDown={handleKeyDown}
    placeholder={placeholder}
    className="flex-1 max-h-32 resize-none min-h-9"
    rows={1}
  />
  <Button
    type="submit"
    size="icon"
    disabled={!input.trim() || isLoading}
    className="shrink-0 h-[38px] w-[38px]"
  >
    {isLoading ? (
      <Loader2 className="size-4 animate-spin" />
    ) : (
      <Send className="size-4" />
    )}
  </Button>
</form>
</div>
);
}

```

File: ./client/src/components/DashboardLayout.tsx

```
import { useAuth } from "@/_core/hooks/useAuth";
import { Avatar, AvatarFallback } from "@/components/ui/avatar";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarHeader,
  SidebarInset,
  SidebarMenu,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarProvider,
  SidebarTrigger,
  useSidebar,
} from "@components/ui/sidebar";
import { getLoginUrl } from "@/const";
import { useIsMobile } from "@/hooks/useMobile";
import { LayoutDashboard, Logout, PanelLeft, Users } from "lucide-react";
import { CSSProperties, useEffect, useRef, useState } from "react";
import { useLocation } from "wouter";
import { DashboardLayoutSkeleton } from "../DashboardLayoutSkeleton";
import { Button } from "../ui/button";

const menuItems = [
  { icon: LayoutDashboard, label: "Page 1", path: "/" },
  { icon: Users, label: "Page 2", path: "/some-path" },
];

const SIDEBAR_WIDTH_KEY = "sidebar-width";
const DEFAULT_WIDTH = 280;
const MIN_WIDTH = 200;
const MAX_WIDTH = 480;

export default function DashboardLayout({
  children,
```

```

}: {
  children: React.ReactNode;
}) {
  const [sidebarWidth, setSidebarWidth] = useState(() => {
    const saved = localStorage.getItem(SIDEBAR_WIDTH_KEY);
    return saved ? parseInt(saved, 10) : DEFAULT_WIDTH;
  });
  const { loading, user } = useAuth();

  useEffect(() => {
    localStorage.setItem(SIDEBAR_WIDTH_KEY, sidebarWidth.toString());
  }, [sidebarWidth]);

  if (loading) {
    return <DashboardLayoutSkeleton />
  }

  if (!user) {
    return (
      <div className="flex items-center justify-center min-h-screen">
        <div className="flex flex-col items-center gap-8 p-8 max-w-md w-full">
          <div className="flex flex-col items-center gap-6">
            <h1 className="text-2xl font-semibold tracking-tight text-center">
              Sign in to continue
            </h1>
            <p className="text-sm text-muted-foreground text-center max-w-sm">
              Access to this dashboard requires authentication. Continue
            </p>
          </div>
          <Button
            onClick={() => {
              window.location.href = getLoginUrl();
            }}
            size="lg"
            className="w-full shadow-lg hover:shadow-xl transition-all"
          >
            Sign in
          </Button>
        </div>
      </div>
    );
  }

  return (

```

```

<SidebarProvider
  style={
    {
      "--sidebar-width": `${sidebarWidth}px`,
    } as CSSProperties
  }
>
  <DashboardLayoutContent setSidebarWidth={setSidebarWidth}>
    {children}
  </DashboardLayoutContent>
</SidebarProvider>
);
}

type DashboardLayoutContentProps = {
  children: React.ReactNode;
  setSidebarWidth: (width: number) => void;
};

function DashboardLayoutContent({
  children,
  setSidebarWidth,
}: DashboardLayoutContentProps) {
  const { user, logout } = useAuth();
  const [location, setLocation] = useLocation();
  const { state, toggleSidebar } = useSidebar();
  const isCollapsed = state === "collapsed";
  const [isResizing, setIsResizing] = useState(false);
  const sidebarRef = useRef<HTMLDivElement>(null);
  const activeMenuItem = menuItems.find(item => item.path === location);
  const isMobile = useIsMobile();

  useEffect(() => {
    if (isCollapsed) {
      setIsResizing(false);
    }
  }, [isCollapsed]);

  useEffect(() => {
    const handleMouseMove = (e: MouseEvent) => {
      if (!isResizing) return;

      const sidebarLeft = sidebarRef.current?.getBoundingClientRect().left;
      const newWidth = e.clientX - sidebarLeft;

```

```

    if (newWidth ≥ MIN_WIDTH && newWidth ≤ MAX_WIDTH) {
      setSidebarWidth(newWidth);
    }
  };

  const handleMouseUp = () ⇒ {
    setIsResizing(false);
  };

  if (isResizing) {
    document.addEventListener("mousemove", handleMouseMove);
    document.addEventListener("mouseup", handleMouseUp);
    document.body.style.cursor = "col-resize";
    document.body.style.userSelect = "none";
  }

  return () ⇒ {
    document.removeEventListener("mousemove", handleMouseMove);
    document.removeEventListener("mouseup", handleMouseUp);
    document.body.style.cursor = "";
    document.body.style.userSelect = "";
  };
}, [isResizing, setSidebarWidth]);

return (
  <
    <div className="relative" ref={sidebarRef}>
      <Sidebar
        collapsible="icon"
        className="border-r-0"
        disableTransition={isResizing}
      >
        <SidebarHeader className="h-16 justify-center">
          <div className="flex items-center gap-3 px-2 transition-all w-100">
            <button
              onClick={toggleSidebar}
              className="h-8 w-8 flex items-center justify-center hover:bg-gray-100"
              aria-label="Toggle navigation"
            >
              <PanelLeft className="h-4 w-4 text-muted-foreground" />
            </button>
            {!isCollapsed ? (
              <div className="flex items-center gap-2 min-w-0">
                <span className="font-semibold tracking-tight truncate">

```

```

        Navigation
      </span>
    </div>
  ) : null}
</div>
</SidebarHeader>

<SidebarContent className="gap-0">
  <SidebarMenu className="px-2 py-1">
    {menuItems.map(item => {
      const isActive = location === item.path;
      return (
        <SidebarMenuItem key={item.path}>
          <SidebarMenuButton
            isActive={isActive}
            onClick={() => setLocation(item.path)}
            tooltip={item.label}
            className={`h-10 transition-all font-normal`}
          >
            <item.icon
              className={`h-4 w-4 ${isActive ? "text-primary" : ""}
            />
            <span>{item.label}</span>
          </SidebarMenuButton>
        </SidebarMenuItem>
      );
    })}
  </SidebarMenu>
</SidebarContent>

<SidebarFooter className="p-3">
  <DropdownMenu>
    <DropdownMenuTrigger asChild>
      <button className="flex items-center gap-3 rounded-lg px-3 py-2">
        <Avatar className="h-9 w-9 border shrink-0">
          <AvatarFallback className="text-xs font-medium">
            {user?.name?.charAt(0).toUpperCase()}
          </AvatarFallback>
        </Avatar>
        <div className="flex-1 min-w-0 group-data-[collapsible=open]:flex-direction:column">
          <p className="text-sm font-medium truncate leading-normal">
            {user?.name || "-"}
          </p>
          <p className="text-xs text-muted-foreground truncate">

```

```

        {user?.email || "-"}
      </p>
    </div>
  </button>
</DropdownMenuTrigger>
<DropdownMenuContent align="end" className="w-48">
  <DropdownMenuItem
    onClick={logout}
    className="cursor-pointer text-destructive focus:text-d
  >
    <LogOut className="mr-2 h-4 w-4" />
    <span>Sign out</span>
  </DropdownMenuItem>
</DropdownMenuContent>
</DropdownMenu>
</SidebarFooter>
</Sidebar>
<div
  className={`absolute top-0 right-0 w-1 h-full cursor-col-resize
  onMouseDown={() => {
    if (isCollapsed) return;
    setIsResizing(true);
  }}
  style={{ zIndex: 50 }}
  />
</div>

<SidebarInset>
  {isMobile && (
    <div className="flex border-b h-14 items-center justify-between"
      <div className="flex items-center gap-2">
        <SidebarTrigger className="h-9 w-9 rounded-lg bg-background
        <div className="flex items-center gap-3">
          <div className="flex flex-col gap-1">
            <span className="tracking-tight text-foreground">
              {activeMenuItem?.label ?? "Menu"}
            </span>
          </div>
        </div>
      </div>
    </div>
  )}
  <main className="flex-1 p-4">{children}</main>
</SidebarInset>

```

```
}  
);  
</>
```

File: ./client/src/components/DashboardLayoutSkeleton.tsx

```
import { Skeleton } from './ui/skeleton';

export function DashboardLayoutSkeleton() {
  return (
    <div className="flex min-h-screen bg-background">
      {/* Sidebar skeleton */}
      <div className="w-[280px] border-r border-border bg-background p-4">
        {/* Logo area */}
        <div className="flex items-center gap-3 px-2">
          <Skeleton className="h-8 w-8 rounded-md" />
          <Skeleton className="h-4 w-24" />
        </div>

        {/* Menu items */}
        <div className="space-y-2 px-2">
          <Skeleton className="h-10 w-full rounded-lg" />
          <Skeleton className="h-10 w-full rounded-lg" />
          <Skeleton className="h-10 w-full rounded-lg" />
        </div>

        {/* User profile area at bottom */}
        <div className="absolute bottom-4 left-4 right-4">
          <div className="flex items-center gap-3 px-1">
            <Skeleton className="h-9 w-9 rounded-full" />
            <div className="flex-1 space-y-2">
              <Skeleton className="h-3 w-20" />
              <Skeleton className="h-2 w-32" />
            </div>
          </div>
        </div>
      </div>

      {/* Main content skeleton */}
      <div className="flex-1 p-4 space-y-4">
        {/* Content blocks */}
        <Skeleton className="h-12 w-48 rounded-lg" />
        <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
          <Skeleton className="h-32 rounded-xl" />
          <Skeleton className="h-32 rounded-xl" />
          <Skeleton className="h-32 rounded-xl" />
        </div>
      </div>
    </div>
  );
}
```

```
        <Skeleton className="h-64 rounded-xl" />
      </div>
    </div>
  );
}
```

File: ./client/src/components/ErrorBoundary.tsx

```
import { cn } from "@/lib/utils";
import { AlertTriangle, RotateCcw } from "lucide-react";
import { Component, ReactNode } from "react";

interface Props {
  children: ReactNode;
}

interface State {
  hasError: boolean;
  error: Error | null;
}

class ErrorBoundary extends Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }

  render() {
    if (this.state.hasError) {
      return (
        <div className="flex items-center justify-center min-h-screen p-8">
          <div className="flex flex-col items-center w-full max-w-2xl p-8">
            <AlertTriangle
              size={48}
              className="text-destructive mb-6 flex-shrink-0"
            />
            <h2 className="text-xl mb-4">An unexpected error occurred.</h2>
            <div className="p-4 w-full rounded bg-muted overflow-auto mb-4">
              <pre className="text-sm text-muted-foreground whitespace-pre-wrap">
                {this.state.error?.stack}
              </pre>
            </div>
          </div>
        </div>
      );
    }
    return this.props.children;
  }
}
```

```

    <button
      onClick={() => window.location.reload()}
      className={cn(
        "flex items-center gap-2 px-4 py-2 rounded-lg",
        "bg-primary text-primary-foreground",
        "hover:opacity-90 cursor-pointer"
      )}
    >
      <RotateCcw size={16} />
      Reload Page
    </button>
  </div>
</div>
);
}

return this.props.children;
}
}

export default ErrorBoundary;

```

File: ./client/src/components/ManusDialog.tsx

```
import { useEffect, useState } from "react";

import { Button } from "@components/ui/button";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogTitle,
} from "@components/ui/dialog";

interface ManusDialogProps {
  title?: string;
  logo?: string;
  open?: boolean;
  onLogin: () => void;
  onOpenChange?: (open: boolean) => void;
  onClose?: () => void;
}

export function ManusDialog({
  title,
  logo,
  open = false,
  onLogin,
  onOpenChange,
  onClose,
}: ManusDialogProps) {
  const [internalOpen, setInternalOpen] = useState(open);

  useEffect(() => {
    if (!onOpenChange) {
      setInternalOpen(open);
    }
  }, [open, onOpenChange]);

  const handleOpenChange = (nextOpen: boolean) => {
    if (onOpenChange) {
      onOpenChange(nextOpen);
    } else {
      setInternalOpen(nextOpen);
    }
  };
}
```



```
        </DialogContent>
    </Dialog>
);
}
```

File: ./client/src/components/Map.tsx

```
/**
 * GOOGLE MAPS FRONTEND INTEGRATION - ESSENTIAL GUIDE
 *
 * USAGE FROM PARENT COMPONENT:
 * =====
 *
 * const mapRef = useRef<google.maps.Map | null>(null);
 *
 * <MapView
 *   initialCenter={{ lat: 40.7128, lng: -74.0060 }}
 *   initialZoom={15}
 *   onMapReady={(map) => {
 *     mapRef.current = map; // Store to control map from parent anytime,
 *   }}
 * </MapView>
 *
 * =====
 * Available Libraries and Core Features:
 * -----
 * 📍 MARKER (from `marker` library)
 * - Attaches to map using { map, position }
 * new google.maps.marker.AdvancedMarkerElement({
 *   map,
 *   position: { lat: 37.7749, lng: -122.4194 },
 *   title: "San Francisco",
 * });
 *
 * -----
 * 🏠 PLACES (from `places` library)
 * - Does not attach directly to map; use data with your map manually.
 * const place = new google.maps.places.Place({ id: PLACE_ID });
 * await place.fetchFields({ fields: ["displayName", "location"] });
 * map.setCenter(place.location);
 * new google.maps.marker.AdvancedMarkerElement({ map, position: place.location });
 *
 * -----
 * 📍 GEOCODER (from `geocoding` library)
 * - Standalone service; manually apply results to map.
 * const geocoder = new google.maps.Geocoder();
 * geocoder.geocode({ address: "New York" }, (results, status) => {
 *   if (status === "OK" && results[0]) {
 *     map.setCenter(results[0].geometry.location);
 *   }
 * });
```

```

*     new google.maps.marker.AdvancedMarkerElement({
*       map,
*       position: results[0].geometry.location,
*     });
*   }
* });
*
* -----
* 📐 GEOMETRY (from `geometry` library)
* - Pure utility functions; not attached to map.
* const dist = google.maps.geometry.spherical.computeDistanceBetween(p1,
*
* -----
* 🗺️ ROUTES (from `routes` library)
* - Combines DirectionsService (standalone) + DirectionsRenderer (map-att
* const directionsService = new google.maps.DirectionsService();
* const directionsRenderer = new google.maps.DirectionsRenderer({ map })
* directionsService.route(
*   { origin, destination, travelMode: "DRIVING" },
*   (res, status) => status === "OK" && directionsRenderer.setDirections
* );
*
* -----
* 🌤️ MAP LAYERS (attach directly to map)
* - new google.maps.TrafficLayer().setMap(map);
* - new google.maps.TransitLayer().setMap(map);
* - new google.maps.BicyclingLayer().setMap(map);
*
* -----
* ✅ SUMMARY
* - "map-attached" → AdvancedMarkerElement, DirectionsRenderer, Layers.
* - "standalone" → Geocoder, DirectionsService, DistanceMatrixService, E
* - "data-only" → Place, Geometry utilities.
*/

```

```

/// <reference types="@types/google.maps" />

```

```

import { useEffect, useRef } from "react";
import { usePersistFn } from "@/hooks/usePersistFn";
import { cn } from "@/lib/utils";

declare global {
  interface Window {
    google?: typeof google;
  }
}

```

```

    }
  }

const API_KEY = import.meta.env.VITE_FRONTEND_FORGE_API_KEY;
const FORGE_BASE_URL =
  import.meta.env.VITE_FRONTEND_FORGE_API_URL ||
  "https://forge.butterfly-effect.dev";
const MAPS_PROXY_URL = `${FORGE_BASE_URL}/v1/maps/proxy`;

function loadMapScript() {
  return new Promise(resolve => {
    const script = document.createElement("script");
    script.src = `${MAPS_PROXY_URL}/maps/api/js?key=${API_KEY}&v=weekly&L:
    script.async = true;
    script.crossOrigin = "anonymous";
    script.onload = () => {
      resolve(null);
      script.remove(); // Clean up immediately
    };
    script.onerror = () => {
      console.error("Failed to load Google Maps script");
    };
    document.head.appendChild(script);
  });
}

interface MapViewProps {
  className?: string;
  initialCenter?: google.maps.LatLngLiteral;
  initialZoom?: number;
  onMapReady?: (map: google.maps.Map) => void;
}

export function MapView({
  className,
  initialCenter = { lat: 37.7749, lng: -122.4194 },
  initialZoom = 12,
  onMapReady,
}: MapViewProps) {
  const mapContainer = useRef<HTMLDivElement>(null);
  const map = useRef<google.maps.Map | null>(null);

  const init = usePersistFn(async () => {
    await loadMapScript();
  });

```

```

    if (!mapContainer.current) {
      console.error("Map container not found");
      return;
    }
    map.current = new window.google.maps.Map(mapContainer.current, {
      zoom: initialZoom,
      center: initialCenter,
      mapTypeControl: true,
      fullscreenControl: true,
      zoomControl: true,
      streetViewControl: true,
      mapId: "DEMO_MAP_ID",
    });
    if (onMapReady) {
      onMapReady(map.current);
    }
  });

  useEffect(() => {
    init();
  }, [init]);

  return (
    <div ref={mapContainer} className={cn("w-full h-[500px]", className)}>
    </div>
  );
}

```

File: ./client/src/const.ts

```
export { COOKIE_NAME, ONE_YEAR_MS } from "@shared/const";

// Generate login URL at runtime so redirect URI reflects the current origin
export const getLoginUrl = () => {
  const oauthPortalUrl = import.meta.env.VITE_OAUTH_PORTAL_URL;
  const appId = import.meta.env.VITE_APP_ID;
  const redirectUri = `${window.location.origin}/api/oauth/callback`;
  const state = btoa(redirectUri);

  const url = new URL(`${oauthPortalUrl}/app-auth`);
  url.searchParams.set("appId", appId);
  url.searchParams.set("redirectUri", redirectUri);
  url.searchParams.set("state", state);
  url.searchParams.set("type", "signIn");

  return url.toString();
};
```

File: ./client/src/contexts/ThemeContext.tsx

```
import React, { createContext, useContext, useEffect, useState } from "react";

type Theme = "light" | "dark";

interface ThemeContextType {
  theme: Theme;
  toggleTheme?: () => void;
  switchable: boolean;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

interface ThemeProviderProps {
  children: React.ReactNode;
  defaultTheme?: Theme;
  switchable?: boolean;
}

export function ThemeProvider({
  children,
  defaultTheme = "light",
  switchable = false,
}: ThemeProviderProps) {
  const [theme, setTheme] = useState<Theme>(() => {
    if (switchable) {
      const stored = localStorage.getItem("theme");
      return (stored as Theme) || defaultTheme;
    }
    return defaultTheme;
  });

  useEffect(() => {
    const root = document.documentElement;
    if (theme === "dark") {
      root.classList.add("dark");
    } else {
      root.classList.remove("dark");
    }

    if (switchable) {
      localStorage.setItem("theme", theme);
    }
  }, [theme, switchable]);
}
```

```

    }
    }, [theme, switchable]));

    const toggleTheme = switchable
      ? () => {
          setTheme(prev => (prev === "light" ? "dark" : "light"));
        }
      : undefined;

    return (
      <ThemeContext.Provider value={{ theme, toggleTheme, switchable }}>
        {children}
      </ThemeContext.Provider>
    );
  }

  export function useTheme() {
    const context = useContext(ThemeContext);
    if (!context) {
      throw new Error("useTheme must be used within ThemeProvider");
    }
    return context;
  }

```

File: ./client/src/hooks/useComposition.ts

```
import { useRef } from "react";
import { usePersistFn } from "../usePersistFn";

export interface UseCompositionReturn<
  T extends HTMLInputElement | HTMLTextAreaElement,
> {
  onCompositionStart: React.CompositionEventHandler<T>;
  onCompositionEnd: React.CompositionEventHandler<T>;
  onKeyDown: React.KeyboardEventHandler<T>;
  isComposing: () => boolean;
}

export interface UseCompositionOptions<
  T extends HTMLInputElement | HTMLTextAreaElement,
> {
  onKeyDown?: React.KeyboardEventHandler<T>;
  onCompositionStart?: React.CompositionEventHandler<T>;
  onCompositionEnd?: React.CompositionEventHandler<T>;
}

type TimerResponse = ReturnType<typeof setTimeout>;

export function useComposition<
  T extends HTMLInputElement | HTMLTextAreaElement = HTMLInputElement,
>(options: UseCompositionOptions<T> = {}): UseCompositionReturn<T> {
  const {
    onKeyDown: originalOnKeyDown,
    onCompositionStart: originalOnCompositionStart,
    onCompositionEnd: originalOnCompositionEnd,
  } = options;

  const c = useRef(false);
  const timer = useRef<TimerResponse | null>(null);
  const timer2 = useRef<TimerResponse | null>(null);

  const onCompositionStart = usePersistFn((e: React.CompositionEvent<T>) :
    if (timer.current) {
      clearTimeout(timer.current);
      timer.current = null;
    }
    if (timer2.current) {
```

```

        clearTimeout(timer2.current);
        timer2.current = null;
    }
    c.current = true;
    originalOnCompositionStart?.(e);
});

const onCompositionEnd = usePersistFn((e: React.CompositionEvent<T>) => {
    // 使用两层 setTimeout 来处理 Safari 浏览器中 compositionEnd 先于 onKeyDown
    timer.current = setTimeout(() => {
        timer2.current = setTimeout(() => {
            c.current = false;
        });
    });
    originalOnCompositionEnd?.(e);
});

const onKeyDown = usePersistFn((e: React.KeyboardEvent<T>) => {
    // 在 composition 状态下, 阻止 ESC 和 Enter (非 shift+Enter) 事件的冒泡
    if (
        c.current &&
        (e.key === "Escape" || (e.key === "Enter" && !e.shiftKey))
    ) {
        e.stopPropagation();
        return;
    }
    originalOnKeyDown?.(e);
});

const isComposing = usePersistFn(() => {
    return c.current;
});

return {
    onCompositionStart,
    onCompositionEnd,
    onKeyDown,
    isComposing,
};
}

```

File: ./client/src/hooks/useMobile.tsx

```
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(
    undefined
  );

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px`);
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    return () => mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}
```

File: ./client/src/hooks/usePersistFn.ts

```
import { useRef } from "react";

type noop = (...args: any[]) => any;

/**
 * usePersistFn instead of useCallback to reduce cognitive load
 */
export function usePersistFn<T extends noop>(fn: T) {
  const fnRef = useRef<T>(fn);
  fnRef.current = fn;

  const persistFn = useRef<T>(null);
  if (!persistFn.current) {
    persistFn.current = function (this: unknown, ...args) {
      return fnRef.current!.apply(this, args);
    } as T;
  }

  return persistFn.current!;
}
```

File: ./client/src/index.css

```
@import "tailwindcss";
@import "tw-animate-css";

/* Import Google Fonts */
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@400;500');

@custom-variant dark (&:is(.dark *));

@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-popover: var(--popover);
  --color-popover-foreground: var(--popover-foreground);
  --color-primary: var(--primary);
  --color-primary-foreground: var(--primary-foreground);
  --color-secondary: var(--secondary);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-muted: var(--muted);
  --color-muted-foreground: var(--muted-foreground);
  --color-accent: var(--accent);
  --color-accent-foreground: var(--accent-foreground);
  --color-destructive: var(--destructive);
  --color-destructive-foreground: var(--destructive-foreground);
  --color-border: var(--border);
  --color-input: var(--input);
  --color-ring: var(--ring);
  --color-chart-1: var(--chart-1);
  --color-chart-2: var(--chart-2);
  --color-chart-3: var(--chart-3);
  --color-chart-4: var(--chart-4);
  --color-chart-5: var(--chart-5);
  --color-sidebar: var(--sidebar);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
```

```
--color-sidebar-accent: var(--sidebar-accent);
--color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
--color-sidebar-border: var(--sidebar-border);
--color-sidebar-ring: var(--sidebar-ring);
}
```

```
:root {
  /* Portfolio Brand Colors */
  --primary: #00d9ff;
  --primary-foreground: #0f1419;
  --accent: #7c3aed;
  --accent-foreground: #f5f5f5;
  --success: #10b981;
  --success-foreground: #f5f5f5;

  /* Dark Theme - Professional */
  --radius: 0.65rem;
  --background: #0f1419;
  --foreground: #f5f5f5;
  --card: #1a202c;
  --card-foreground: #f5f5f5;
  --popover: #1a202c;
  --popover-foreground: #f5f5f5;
  --secondary: #2d3748;
  --secondary-foreground: #a0aec0;
  --muted: #4a5568;
  --muted-foreground: #a0aec0;
  --destructive: #ef4444;
  --destructive-foreground: #f5f5f5;
  --border: #2d3748;
  --input: #2d3748;
  --ring: #00d9ff;
  --sidebar: #1a202c;
  --sidebar-foreground: #f5f5f5;
  --sidebar-accent: #00d9ff;
  --sidebar-accent-foreground: #0f1419;
  --sidebar-border: #2d3748;
  --sidebar-ring: #00d9ff;

  /* Chart colors */
  --chart-1: #00d9ff;
  --chart-2: #7c3aed;
  --chart-3: #10b981;
  --chart-4: #f59e0b;
}
```

```

--chart-5: #ef4444;
}

.dark {
  /* Dark theme is default for this portfolio */
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }

  body {
    @apply bg-background text-foreground;
    font-family: Inter, sans-serif;
  }

  h1, h2, h3, h4, h5, h6 {
    @apply font-bold;
    font-family: Poppins, sans-serif;
  }

  button:not(:disabled),
  [role="button"]:not([aria-disabled="true"]),
  [type="button"]:not(:disabled),
  [type="submit"]:not(:disabled),
  [type="reset"]:not(:disabled),
  a[href],
  select:not(:disabled),
  input[type="checkbox"]:not(:disabled),
  input[type="radio"]:not(:disabled) {
    @apply cursor-pointer;
  }
}

@layer components {
  /**
   * Custom container utility that centers content and adds responsive padding
   */
  .container {
    width: 100%;
    margin-left: auto;
    margin-right: auto;
    padding-left: 1rem;
  }
}

```

```
padding-right: 1rem;
}

.flex {
  min-height: 0;
  min-width: 0;
}

@media (min-width: 640px) {
  .container {
    padding-left: 1.5rem;
    padding-right: 1.5rem;
  }
}

@media (min-width: 1024px) {
  .container {
    padding-left: 2rem;
    padding-right: 2rem;
    max-width: 1280px;
  }
}

/* Typography System */
.font-display {
  font-family: Poppins, sans-serif;
  @apply font-bold;
}

.font-body {
  font-family: Inter, sans-serif;
  @apply font-normal;
}

.font-mono {
  font-family: 'Fira Code', monospace;
}

/* Gradient Accent */
.gradient-accent {
  @apply bg-gradient-to-r from-cyan-400 to-purple-500 bg-clip-text text-
}

/* Glow Effect */
```

```

.glow-cyan {
  @apply shadow-lg shadow-cyan-500/20;
}

.glow-purple {
  @apply shadow-lg shadow-purple-500/20;
}
} ````

## File: ./client/src/lib/trpc.ts
````typescript
import { createTRPCReact } from "@trpc/react-query";
import type { AppRouter } from "../../server/routers";

export const trpc = createTRPCReact<AppRouter>();

```

## File: ./client/src/lib/utils.ts

```

import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}

```

## File: ./client/src/main.tsx

```
import { trpc } from "@lib/trpc";
import { UNAUTHED_ERR_MSG } from '@shared/const';
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { httpBatchLink, TRPCClientError } from "@trpc/client";
import { createRoot } from "react-dom/client";
import superjson from "superjson";
import App from "./App";
import { getLoginUrl } from "./const";
import "./index.css";

const queryClient = new QueryClient();

const redirectToLoginIfUnauthorized = (error: unknown) => {
 if (!(error instanceof TRPCClientError)) return;
 if (typeof window === "undefined") return;

 const isUnauthorized = error.message === UNAUTHED_ERR_MSG;

 if (!isUnauthorized) return;

 window.location.href = getLoginUrl();
};

queryClient.getQueryCache().subscribe(event => {
 if (event.type === "updated" && event.action.type === "error") {
 const error = event.query.state.error;
 redirectToLoginIfUnauthorized(error);
 console.error("[API Query Error]", error);
 }
});

queryClient.getMutationCache().subscribe(event => {
 if (event.type === "updated" && event.action.type === "error") {
 const error = event.mutation.state.error;
 redirectToLoginIfUnauthorized(error);
 console.error("[API Mutation Error]", error);
 }
});

const trpcClient = trpc.createClient({
 links: [
```

```

 httpBatchLink({
 url: "/api/trpc",
 transformer: superjson,
 fetch(input, init) {
 return globalThis.fetch(input, {
 ...(init ?? {}),
 credentials: "include",
 });
 },
 }),
],
});

createRoot(document.getElementById("root")!).render(
 <trpc.Provider client={trpcClient} queryClient={queryClient}>
 <QueryClientProvider client={queryClient}>
 <App />
 </QueryClientProvider>
 </trpc.Provider>
);

```

## File: ./client/src/pages/ComponentShowcase.tsx

```
import {
 Accordion,
 AccordionContent,
 AccordionItem,
 AccordionTrigger,
} from "@components/ui/accordion";
import { Alert, AlertDescription, AlertTitle } from "@components/ui/alert";
import { AspectRatio } from "@components/ui/aspect-ratio";
import { Avatar, AvatarFallback, AvatarImage } from "@components/ui/avatar";
import { Badge } from "@components/ui/badge";
import {
 Breadcrumb,
 BreadcrumbItem,
 BreadcrumbLink,
 BreadcrumbList,
 BreadcrumbPage,
 BreadcrumbSeparator,
} from "@components/ui/breadcrumb";
import { Button } from "@components/ui/button";
import { Calendar } from "@components/ui/calendar";
import {
 Card,
 CardContent,
 CardDescription,
 CardFooter,
 CardHeader,
 CardTitle,
} from "@components/ui/card";
import {
 Carousel,
 CarouselContent,
 CarouselItem,
 CarouselNext,
 CarouselPrevious,
} from "@components/ui/carousel";
import { Checkbox } from "@components/ui/checkbox";
import {
 Collapsible,
 CollapsibleContent,
 CollapsibleTrigger,
} from "@components/ui/collapsible";
```

```
import {
 Command,
 CommandEmpty,
 CommandGroup,
 CommandInput,
 CommandItem,
 CommandList,
} from "@components/ui/command";
import {
 ContextMenu,
 ContextMenuContent,
 ContextMenuItem,
 ContextMenuTrigger,
} from "@components/ui/context-menu";
import {
 Dialog,
 DialogContent,
 DialogDescription,
 DialogHeader,
 DialogTitle,
 DialogTrigger,
} from "@components/ui/dialog";
import {
 Drawer,
 DrawerClose,
 DrawerContent,
 DrawerDescription,
 DrawerFooter,
 DrawerHeader,
 DrawerTitle,
 DrawerTrigger,
} from "@components/ui/drawer";
import {
 DropdownMenu,
 DropdownMenuContent,
 DropdownMenuItem,
 DropdownMenuLabel,
 DropdownMenuSeparator,
 DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import {
 HoverCard,
 HoverCardContent,
 HoverCardTrigger,
```

```
} from "@components/ui/hover-card";
import { Input } from "@components/ui/input";
import {
 InputOTP,
 InputOTPGroup,
 InputOTPSlot,
} from "@components/ui/input-otp";
import { Label } from "@components/ui/label";
import {
 Menubar,
 MenubarContent,
 MenubarItem,
 MenubarMenu,
 MenubarSeparator,
 MenubarTrigger,
} from "@components/ui/menubar";
import {
 Pagination,
 PaginationContent,
 PaginationItem,
 PaginationLink,
 PaginationNext,
 PaginationPrevious,
} from "@components/ui/pagination";
import {
 Popover,
 PopoverContent,
 PopoverTrigger,
} from "@components/ui/popover";
import { Progress } from "@components/ui/progress";
import { RadioGroup, RadioGroupItem } from "@components/ui/radio-group";
import {
 ResizableHandle,
 ResizablePanel,
 ResizablePanelGroup,
} from "@components/ui/resizable";
import { ScrollArea } from "@components/ui/scroll-area";
import {
 Select,
 SelectContent,
 SelectItem,
 SelectTrigger,
 SelectValue,
} from "@components/ui/select";
```

```
import { Separator } from "@components/ui/separator";
import {
 Sheet,
 SheetContent,
 SheetDescription,
 SheetHeader,
 SheetTitle,
 SheetTrigger,
} from "@components/ui/sheet";
import { Skeleton } from "@components/ui/skeleton";
import { Slider } from "@components/ui/slider";
import { Switch } from "@components/ui/switch";
import {
 Table,
 TableBody,
 TableCaption,
 TableCell,
 TableHead,
 TableHeader,
 TableRow,
} from "@components/ui/table";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
import { Textarea } from "@components/ui/textarea";
import { Toggle } from "@components/ui/toggle";
import { ToggleGroup, ToggleGroupItem } from "@components/ui/toggle-group";
import {
 Tooltip,
 TooltipContent,
 TooltipTrigger,
} from "@components/ui/tooltip";
import { useTheme } from "@contexts/ThemeContext";
import { format } from "date-fns";
import { zhCN } from "date-fns/locale";
import {
 AlertCircle,
 CalendarIcon,
 Check,
 Clock,
 Moon,
 Sun,
 X,
} from "lucide-react";
import { useState } from "react";
import { toast as sonnerToast } from "sonner";
```

```

import { AIChatBox, type Message } from "@components/AIChatBox";

export default function ComponentsShowcase() {
 const { theme, toggleTheme } = useTheme();
 const [date, setDate] = useState<Date | undefined>(new Date());
 const [datePickerDate, setDatePickerDate] = useState<Date>();
 const [selectedFruits, setSelectedFruits] = useState<string[]>([]);
 const [progress, setProgress] = useState(33);
 const [currentPage, setCurrentPage] = useState(2);
 const [openCombobox, setOpenCombobox] = useState(false);
 const [selectedFramework, setSelectedFramework] = useState("");
 const [selectedMonth, setSelectedMonth] = useState("");
 const [selectedYear, setSelectedYear] = useState("");
 const [dialogInput, setDialogInput] = useState("");
 const [dialogOpen, setDialogOpen] = useState(false);

 // AI ChatBox demo state
 const [chatMessages, setChatMessages] = useState<Message[]>([
 { role: "system", content: "You are a helpful assistant." },
]);
 const [isChatLoading, setIsChatLoading] = useState(false);

 const handleDialogSubmit = () => {
 console.log("Dialog submitted with value:", dialogInput);
 sonnerToast.success("Submitted successfully", {
 description: `Input: ${dialogInput}`,
 });
 setDialogInput("");
 setDialogOpen(false);
 };

 const handleDialogKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) :
 if (e.key === "Enter" && !e.nativeEvent.isComposing) {
 e.preventDefault();
 handleDialogSubmit();
 }
 };

 const handleChatSend = (content: string) => {
 // Add user message
 const newMessages: Message[] = [...chatMessages, { role: "user", content: content }];
 setChatMessages(newMessages);

 // Simulate AI response with delay
 };

```

```

setIsChatLoading(true);
setTimeout(() => {
 const aiResponse: Message = {
 role: "assistant",
 content: `This is a **demo response**. In a real app, you would c
 };
 setChatMessages([...newMessages, aiResponse]);
 setIsChatLoading(false);
}, 1500);
};

```

```

return (
 <div className="min-h-screen bg-background text-foreground">
 <main className="container max-w-6xl mx-auto">
 <div className="space-y-2 justify-between flex">
 <h2 className="text-3xl font-bold tracking-tight mb-6">
 Shadcn/ui Component Library
 </h2>
 <Button variant="outline" size="icon" onClick={toggleTheme}>
 {theme === "light" ? (
 <Moon className="h-5 w-5" />
) : (
 <Sun className="h-5 w-5" />
)}
 </Button>
 </div>

 <div className="space-y-12">
 {/* Text Colors Section */}
 <section className="space-y-4">
 <h3 className="text-2xl font-semibold">Text Colors</h3>
 <Card>
 <CardContent className="pt-6">
 <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
 <div className="space-y-3">
 <div>
 <p className="text-sm text-muted-foreground mb-1">
 Foreground (Default)
 </p>
 <p className="text-foreground text-lg">
 Default text color for main content
 </p>
 </div>
 </div>
 </div>
 </CardContent>
 </Card>
 </section>
 </div>
 </main>
 </div>
)

```

```
<p className="text-sm text-muted-foreground mb-1">
 Muted Foreground
</p>
<p className="text-muted-foreground text-lg">
 Muted text for secondary information
</p>
</div>
<div>
 <p className="text-sm text-muted-foreground mb-1">
 Primary
 </p>
 <p className="text-primary text-lg font-medium">
 Primary brand color text
 </p>
</div>
<div>
 <p className="text-sm text-muted-foreground mb-1">
 Secondary Foreground
 </p>
 <p className="text-secondary-foreground text-lg">
 Secondary action text color
 </p>
</div>
</div>
<div className="space-y-3">
 <div>
 <p className="text-sm text-muted-foreground mb-1">
 Accent Foreground
 </p>
 <p className="text-accent-foreground text-lg">
 Accent text for emphasis
 </p>
 </div>
 <div>
 <p className="text-sm text-muted-foreground mb-1">
 Destructive
 </p>
 <p className="text-destructive text-lg font-medium">
 Error or destructive action text
 </p>
 </div>
 <div>
 <p className="text-sm text-muted-foreground mb-1">
 Card Foreground
```

```

 </p>
 <p className="text-card-foreground text-lg">
 Text color on card backgrounds
 </p>
 </div>
 <div>
 <p className="text-sm text-muted-foreground mb-1">
 Popover Foreground
 </p>
 <p className="text-popover-foreground text-lg">
 Text color in popovers
 </p>
 </div>
</div>
</CardContent>
</Card>
</section>

{/* Color Combinations Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Color Combinations</h3>
 <Card>
 <CardContent className="pt-6">
 <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-co
 <div className="bg-primary text-primary-foreground round
 <p className="font-medium mb-1">Primary</p>
 <p className="text-sm opacity-90">
 Primary background with foreground text
 </p>
 </div>
 <div className="bg-secondary text-secondary-foreground
 <p className="font-medium mb-1">Secondary</p>
 <p className="text-sm opacity-90">
 Secondary background with foreground text
 </p>
 </div>
 <div className="bg-muted text-muted-foreground rounded-
 <p className="font-medium mb-1">Muted</p>
 <p className="text-sm opacity-90">
 Muted background with foreground text
 </p>
 </div>
 <div className="bg-accent text-accent-foreground rounder

```

```

 <p className="font-medium mb-1">Accent</p>
 <p className="text-sm opacity-90">
 Accent background with foreground text
 </p>
 </div>
 <div className="bg-destructive text-destructive-foreground">
 <p className="font-medium mb-1">Destructive</p>
 <p className="text-sm opacity-90">
 Destructive background with foreground text
 </p>
 </div>
 <div className="bg-card text-card-foreground rounded-lg">
 <p className="font-medium mb-1">Card</p>
 <p className="text-sm opacity-90">
 Card background with foreground text
 </p>
 </div>
 <div className="bg-popover text-popover-foreground rounded-lg">
 <p className="font-medium mb-1">Popover</p>
 <p className="text-sm opacity-90">
 Popover background with foreground text
 </p>
 </div>
 <div className="bg-background text-foreground rounded-lg">
 <p className="font-medium mb-1">Background</p>
 <p className="text-sm opacity-90">
 Default background with foreground text
 </p>
 </div>
</div>
</CardContent>
</Card>
</section>

{/* Buttons Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Buttons</h3>
 <Card>
 <CardContent className="pt-6">
 <div className="flex flex-wrap gap-4">
 <Button>Default</Button>
 <Button variant="secondary">Secondary</Button>
 <Button variant="destructive">Destructive</Button>
 <Button variant="outline">Outline</Button>
 </div>
 </CardContent>
 </Card>
</section>

```

```

 <Button variant="ghost">Ghost</Button>
 <Button variant="link">Link</Button>
 <Button size="sm">Small</Button>
 <Button size="lg">Large</Button>
 <Button size="icon">
 <Check className="h-4 w-4" />
 </Button>
 </div>
</CardContent>
</Card>
</section>

{/* Form Inputs Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Form Inputs</h3>
 <Card>
 <CardContent className="pt-6 space-y-6">
 <div className="space-y-2">
 <Label htmlFor="email">Email</Label>
 <Input id="email" type="email" placeholder="Email" />
 </div>
 <div className="space-y-2">
 <Label htmlFor="message">Message</Label>
 <Textarea
 id="message"
 placeholder="Type your message here."
 />
 </div>
 <div className="space-y-2">
 <Label>Select</Label>
 <Select>
 <SelectTrigger>
 <SelectValue placeholder="Select a fruit" />
 </SelectTrigger>
 <SelectContent>
 <SelectItem value="apple">Apple</SelectItem>
 <SelectItem value="banana">Banana</SelectItem>
 <SelectItem value="orange">Orange</SelectItem>
 </SelectContent>
 </Select>
 </div>
 <div className="flex items-center space-x-2">
 <Checkbox id="terms" />
 <Label htmlFor="terms">Accept terms and conditions</Label>
 </div>
 </CardContent>
 </Card>
</section>

```

```

</div>
<div className="flex items-center space-x-2">
 <Switch id="airplane-mode" />
 <Label htmlFor="airplane-mode">Airplane Mode</Label>
</div>
<div className="space-y-2">
 <Label>Radio Group</Label>
 <RadioGroup defaultValue="option-one">
 <div className="flex items-center space-x-2">
 <RadioGroupItem value="option-one" id="option-one">
 <Label htmlFor="option-one">Option One</Label>
 </div>
 <div className="flex items-center space-x-2">
 <RadioGroupItem value="option-two" id="option-two">
 <Label htmlFor="option-two">Option Two</Label>
 </div>
 </RadioGroup>
 </div>
 <div className="space-y-2">
 <Label>Slider</Label>
 <Slider defaultValue={[50]} max={100} step={1} />
 </div>
 <div className="space-y-2">
 <Label>Input OTP</Label>
 <InputOTP maxLength={6}>
 <InputOTPGroup>
 <InputOTPSlot index={0} />
 <InputOTPSlot index={1} />
 <InputOTPSlot index={2} />
 <InputOTPSlot index={3} />
 <InputOTPSlot index={4} />
 <InputOTPSlot index={5} />
 </InputOTPGroup>
 </InputOTP>
 </div>
 <div className="space-y-2">
 <Label>Date Time Picker</Label>
 <Popover>
 <PopoverTrigger asChild>
 <Button
 variant="outline"
 className={`w-full justify-start text-left font-n
 !datePickerDate && "text-muted-foreground"
 `}
 >

```



```

</Popover>
{datePickerDate && (
 <p className="text-sm text-muted-foreground">
 Selected:{" "}
 {format(datePickerDate, "yyyy/MM/dd HH:mm", {
 locale: zhCN,
 })}
 </p>
)}
</div>
<div className="space-y-2">
 <Label>Searchable Dropdown</Label>
 <Popover open={openCombobox} onOpenChange={setOpenCombobox}>
 <PopoverTrigger asChild>
 <Button
 variant="outline"
 role="combobox"
 aria-expanded={openCombobox}
 className="w-full justify-between"
 >
 {selectedFramework
 ? [
 { value: "react", label: "React" },
 { value: "vue", label: "Vue" },
 { value: "angular", label: "Angular" },
 { value: "svelte", label: "Svelte" },
 { value: "nextjs", label: "Next.js" },
 { value: "nuxt", label: "Nuxt" },
 { value: "remix", label: "Remix" },
].find(fw => fw.value === selectedFramework)?
 : "Select framework..."}
 <CalendarIcon className="ml-2 h-4 w-4 shrink-0 opacity-50" />
 </Button>
 </PopoverTrigger>
 <PopoverContent className="w-full p-0">
 <Command>
 <CommandInput placeholder="Search frameworks..." />
 <CommandList>
 <CommandEmpty>No framework found</CommandEmpty>
 <CommandGroup>
 {[
 { value: "react", label: "React" },
 { value: "vue", label: "Vue" },
 { value: "angular", label: "Angular" },

```

```

 { value: "svelte", label: "Svelte" },
 { value: "nextjs", label: "Next.js" },
 { value: "nuxt", label: "Nuxt" },
 { value: "remix", label: "Remix" },
].map(framework => (
 <CommandItem
 key={framework.value}
 value={framework.value}
 onSelect={currentValue => {
 setSelectedFramework(
 currentValue === selectedFramework
 ? ""
 : currentValue
);
 setOpenCombobox(false);
 }}
 >
 <Check
 className={`mr-2 h-4 w-4 ${
 selectedFramework === framework.value
 ? "opacity-100"
 : "opacity-0"
 }`}
 />
 {framework.label}
 </CommandItem>
))}
</CommandGroup>
</CommandList>
</Command>
</PopoverContent>
</Popover>
{selectedFramework && (
 <p className="text-sm text-muted-foreground">
 Selected:{" "}
 {
 [
 { value: "react", label: "React" },
 { value: "vue", label: "Vue" },
 { value: "angular", label: "Angular" },
 { value: "svelte", label: "Svelte" },
 { value: "nextjs", label: "Next.js" },
 { value: "nuxt", label: "Nuxt" },
 { value: "remix", label: "Remix" },

```

```

].find(fw => fw.value === selectedFramework)?.label
 }
 }
</p>
})
</div>
<div className="space-y-2">
 <div className="grid grid-cols-2 gap-4">
 <div className="space-y-2">
 <Label htmlFor="month" className="text-sm font-medium">
 Month
 </Label>
 <Select
 value={selectedMonth}
 onChange={setSelectedMonth}
 >
 <SelectTrigger id="month">
 <SelectValue placeholder="MM" />
 </SelectTrigger>
 <SelectContent>
 {Array.from({ length: 12 }, (_, i) => i + 1).map(
 month => (
 <SelectItem
 key={month}
 value={month.toString().padStart(2, "0")}
 >
 {month.toString().padStart(2, "0")}
 </SelectItem>
)
)}
 </SelectContent>
 </Select>
 </div>
 <div className="space-y-2">
 <Label htmlFor="year" className="text-sm font-medium">
 Year
 </Label>
 <Select
 value={selectedYear}
 onChange={setSelectedYear}
 >
 <SelectTrigger id="year">
 <SelectValue placeholder="YYYY" />
 </SelectTrigger>
 <SelectContent>

```

```

 {Array.from(
 { length: 10 },
 (_, i) => new Date().getFullYear() - 5 + i
).map(year => (
 <SelectItem key={year} value={year.toString()}
 {year}
 </SelectItem>
))}
 </SelectContent>
 </Select>
 </div>
</div>
{selectedMonth && selectedYear && (
 <p className="text-sm text-muted-foreground">
 Selected: {selectedYear}/{selectedMonth}/
 </p>
)}
</div>
</CardContent>
</Card>
</section>

{/* Data Display Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Data Display</h3>
 <Card>
 <CardContent className="pt-6 space-y-6">
 <div className="space-y-2">
 <Label>Badges</Label>
 <div className="flex flex-wrap gap-2">
 <Badge>Default</Badge>
 <Badge variant="secondary">Secondary</Badge>
 <Badge variant="destructive">Destructive</Badge>
 <Badge variant="outline">Outline</Badge>
 </div>
 </div>
 <Separator />
 <div className="space-y-2">
 <Label>Avatar</Label>
 <div className="flex gap-4">
 <Avatar>
 <AvatarImage src="https://github.com/shadcn.png" />
 <AvatarFallback>CN</AvatarFallback>
 </Avatar>

```

```

 <Avatar>
 <AvatarFallback>AB</AvatarFallback>
 </Avatar>
 </div>
 </div>
 <Separator />
 <div className="space-y-2">
 <Label>Progress</Label>
 <Progress value={progress} />
 <div className="flex gap-2">
 <Button
 size="sm"
 onClick={() => setProgress(Math.max(0, progress - 10))}
 >
 -10
 </Button>
 <Button
 size="sm"
 onClick={() => setProgress(Math.min(100, progress + 10))}
 >
 +10
 </Button>
 </div>
 </div>
 <Separator />
 <div className="space-y-2">
 <Label>Skeleton</Label>
 <div className="space-y-2">
 <Skeleton className="h-4 w-full" />
 <Skeleton className="h-4 w-3/4" />
 <Skeleton className="h-4 w-1/2" />
 </div>
 </div>
 <Separator />
 <div className="space-y-2">
 <Label>Pagination</Label>
 <Pagination>
 <PaginationContent>
 <PaginationItem>
 <PaginationPrevious
 href="#"
 onClick={e => {
 e.preventDefault();
 setCurrentPage(Math.max(1, currentPage - 1));
 }}
 />
 </PaginationItem>
 </PaginationContent>
 </Pagination>
 </div>
 </div>
)

```



```

</TableHeader>
<TableBody>
 <TableRow>
 <TableCell className="font-medium">INV001</TableCell>
 <TableCell>Paid</TableCell>
 <TableCell>Credit Card</TableCell>
 <TableCell className="text-right">$250.00</TableCell>
 </TableRow>
 <TableRow>
 <TableCell className="font-medium">INV002</TableCell>
 <TableCell>Pending</TableCell>
 <TableCell>PayPal</TableCell>
 <TableCell className="text-right">$150.00</TableCell>
 </TableRow>
 <TableRow>
 <TableCell className="font-medium">INV003</TableCell>
 <TableCell>Unpaid</TableCell>
 <TableCell>Bank Transfer</TableCell>
 <TableCell className="text-right">$350.00</TableCell>
 </TableRow>
</TableBody>
</Table>
</div>
<Separator />
<div className="space-y-2">
 <Label>Menubar</Label>
 <Menubar>
 <MenubarMenu>
 <MenubarTrigger>File</MenubarTrigger>
 <MenubarContent>
 <MenubarItem>New Tab</MenubarItem>
 <MenubarItem>New Window</MenubarItem>
 <MenubarSeparator />
 <MenubarItem>Share</MenubarItem>
 <MenubarSeparator />
 <MenubarItem>Print</MenubarItem>
 </MenubarContent>
 </MenubarMenu>
 <MenubarMenu>
 <MenubarTrigger>Edit</MenubarTrigger>
 <MenubarContent>
 <MenubarItem>Undo</MenubarItem>
 <MenubarItem>Redo</MenubarItem>
 </MenubarContent>
 </MenubarMenu>
 </Menubar>
</div>

```

```

 </MenubarMenu>
 <MenubarMenu>
 <MenubarTrigger>View</MenubarTrigger>
 <MenubarContent>
 <MenubarItem>Reload</MenubarItem>
 <MenubarItem>Force Reload</MenubarItem>
 </MenubarContent>
 </MenubarMenu>
 </Menubar>
 </div>
 <Separator />
 <div className="space-y-2">
 <Label>Breadcrumbs</Label>
 <Breadcrumbs>
 <BreadcrumbsList>
 <BreadcrumbsItem>
 <BreadcrumbsLink href="/">Home</BreadcrumbsLink>
 </BreadcrumbsItem>
 <BreadcrumbsSeparator />
 <BreadcrumbsItem>
 <BreadcrumbsLink href="/components">
 Components
 </BreadcrumbsLink>
 </BreadcrumbsItem>
 <BreadcrumbsSeparator />
 <BreadcrumbsItem>
 <BreadcrumbsPage>Breadcrumbs</BreadcrumbsPage>
 </BreadcrumbsItem>
 </BreadcrumbsList>
 </Breadcrumbs>
 </div>
 </CardContent>
</Card>
</section>

{/* Alerts Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Alerts</h3>
 <div className="space-y-4">
 <Alert>
 <AlertCircle className="h-4 w-4" />
 <AlertTitle>Heads up!</AlertTitle>
 <AlertDescription>
 You can add components to your app using the cli.
 </AlertDescription>
 </Alert>
 </div>
</section>

```

```

 </AlertDescription>
 </Alert>
 <Alert variant="destructive">
 <X className="h-4 w-4" />
 <AlertTitle>Error</AlertTitle>
 <AlertDescription>
 Your session has expired. Please log in again.
 </AlertDescription>
 </Alert>
</div>
</section>

{ /* Tabs Section */ }
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Tabs</h3>
 <Tabs defaultValue="account" className="w-full">
 <TabsList className="grid w-full grid-cols-3">
 <TabsTrigger value="account">Account</TabsTrigger>
 <TabsTrigger value="password">Password</TabsTrigger>
 <TabsTrigger value="settings">Settings</TabsTrigger>
 </TabsList>
 <TabsContent value="account">
 <Card>
 <CardHeader>
 <CardTitle>Account</CardTitle>
 <CardDescription>
 Make changes to your account here.
 </CardDescription>
 </CardHeader>
 <CardContent className="space-y-2">
 <div className="space-y-1">
 <Label htmlFor="name">Name</Label>
 <Input id="name" defaultValue="Pedro Duarte" />
 </div>
 </CardContent>
 <CardFooter>
 <Button>Save changes</Button>
 </CardFooter>
 </Card>
 </TabsContent>
 <TabsContent value="password">
 <Card>
 <CardHeader>
 <CardTitle>Password</CardTitle>

```

```

 <CardDescription>
 Change your password here.
 </CardDescription>
 </CardHeader>
 <CardContent className="space-y-2">
 <div className="space-y-1">
 <Label htmlFor="current">Current password</Label>
 <Input id="current" type="password" />
 </div>
 <div className="space-y-1">
 <Label htmlFor="new">New password</Label>
 <Input id="new" type="password" />
 </div>
 </CardContent>
 <CardFooter>
 <Button>Save password</Button>
 </CardFooter>
</Card>
</TabsContent>
<TabsContent value="settings">
 <Card>
 <CardHeader>
 <CardTitle>Settings</CardTitle>
 <CardDescription>
 Manage your settings here.
 </CardDescription>
 </CardHeader>
 <CardContent>
 <p className="text-sm text-muted-foreground">
 Settings content goes here.
 </p>
 </CardContent>
 </Card>
</TabsContent>
</Tabs>
</section>

{/* Accordion Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Accordion</h3>
 <Accordion type="single" collapsible className="w-full">
 <AccordionItem value="item-1">
 <AccordionTrigger>Is it accessible?</AccordionTrigger>
 <AccordionContent>

```

```

 Yes. It adheres to the WAI-ARIA design pattern.
 </AccordionContent>
 </AccordionItem>
 <AccordionItem value="item-2">
 <AccordionTrigger>Is it styled?</AccordionTrigger>
 <AccordionContent>
 Yes. It comes with default styles that matches the other
 components' aesthetic.
 </AccordionContent>
 </AccordionItem>
 <AccordionItem value="item-3">
 <AccordionTrigger>Is it animated?</AccordionTrigger>
 <AccordionContent>
 Yes. It's animated by default, but you can disable it if
 you prefer.
 </AccordionContent>
 </AccordionItem>
 </Accordion>
</section>

{/* Collapsible Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Collapsible</h3>
 <Collapsible>
 <Card>
 <CardHeader>
 <CollapsibleTrigger asChild>
 <Button variant="ghost" className="w-full justify-between">
 <CardTitle>@peduarte starred 3 repositories</CardTitle>
 </Button>
 </CollapsibleTrigger>
 </CardHeader>
 <CollapsibleContent>
 <CardContent>
 <div className="space-y-2">
 <div className="rounded-md border px-4 py-3 font-mono text-sm">
 @radix-ui/primitives
 </div>
 <div className="rounded-md border px-4 py-3 font-mono text-sm">
 @radix-ui/colors
 </div>
 <div className="rounded-md border px-4 py-3 font-mono text-sm">
 @stitches/react
 </div>
 </div>
 </CardContent>
 </CollapsibleContent>
 </Card>
 </Collapsible>
</section>

```



```

 <Button onClick={handleDialogSubmit}>Submit</Button>
 </div>
 </DialogContent>
 </Dialog>

 <Sheet>
 <SheetTrigger asChild>
 <Button variant="outline">Open Sheet</Button>
 </SheetTrigger>
 <SheetContent>
 <SheetHeader>
 <SheetTitle>Edit profile</SheetTitle>
 <SheetDescription>
 Make changes to your profile here. Click save when
 you're done.
 </SheetDescription>
 </SheetHeader>
 </SheetContent>
 </Sheet>

 <Drawer>
 <DrawerTrigger asChild>
 <Button variant="outline">Open Drawer</Button>
 </DrawerTrigger>
 <DrawerContent>
 <DrawerHeader>
 <DrawerTitle>Are you absolutely sure?</DrawerTitle>
 <DrawerDescription>
 This action cannot be undone.
 </DrawerDescription>
 </DrawerHeader>
 <DrawerFooter>
 <Button>Submit</Button>
 <DrawerClose asChild>
 <Button variant="outline">Cancel</Button>
 </DrawerClose>
 </DrawerFooter>
 </DrawerContent>
 </Drawer>

 <Popover>
 <PopoverTrigger asChild>
 <Button variant="outline">Open Popover</Button>
 </PopoverTrigger>

```





```

{ /* Carousel Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Carousel</h3>
 <Card>
 <CardContent className="pt-6">
 <Carousel className="w-full max-w-xs mx-auto">
 <CarouselContent>
 {Array.from({ length: 5 }).map((_, index) => (
 <CarouselItem key={index}>
 <div className="p-1">
 <Card>
 <CardContent className="flex aspect-square it

 {index + 1}

 </CardContent>
 </Card>
 </div>
 </CarouselItem>
))}
 </CarouselContent>
 <CarouselPrevious />
 <CarouselNext />
 </Carousel>
 </CardContent>
 </Card>
</section>

{ /* Toggle Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Toggle</h3>
 <Card>
 <CardContent className="pt-6 space-y-4">
 <div className="space-y-2">
 <Label>Toggle</Label>
 <div className="flex gap-2">
 <Toggle aria-label="Toggle italic">
 B
 </Toggle>
 <Toggle aria-label="Toggle italic">
 I
 </Toggle>
 <Toggle aria-label="Toggle underline">

```



```

 <div className="space-y-4">
 {Array.from({ length: 20 }).map((_, i) => (
 <div key={i} className="text-sm">
 Item {i + 1}: This is a scrollable content area
 </div>
))}
 </div>
 </div>
 </ScrollArea>
 </div>
</CardContent>
</Card>
</section>

```

```

{/* Resizable Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Resizable Panels</h3>
 <Card>
 <CardContent className="pt-6">
 <ResizablePanelGroup
 direction="horizontal"
 className="min-h-[200px] rounded-lg border"
 >
 <ResizablePanel defaultSize={50}>
 <div className="flex h-full items-center justify-center">
 Panel One
 </div>
 </ResizablePanel>
 <ResizableHandle />
 <ResizablePanel defaultSize={50}>
 <div className="flex h-full items-center justify-center">
 Panel Two
 </div>
 </ResizablePanel>
 </ResizablePanelGroup>
 </CardContent>
 </Card>
</section>

```

```

{/* Toast Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">Toast</h3>
 <Card>
 <CardContent className="pt-6 space-y-4">

```

```

<div className="space-y-2">
 <Label>Sonner Toast</Label>
 <div className="flex flex-wrap gap-2">
 <Button
 variant="outline"
 onClick={() => {
 sonnerToast.success("Operation successful", {
 description: "Your changes have been saved",
 });
 }}
 >
 Success
 </Button>
 <Button
 variant="outline"
 onClick={() => {
 sonnerToast.error("Operation failed", {
 description:
 "Cannot complete operation, please try again"
 });
 }}
 >
 Error
 </Button>
 <Button
 variant="outline"
 onClick={() => {
 sonnerToast.info("Information", {
 description: "This is an information message",
 });
 }}
 >
 Info
 </Button>
 <Button
 variant="outline"
 onClick={() => {
 sonnerToast.warning("Warning", {
 description:
 "Please note the impact of this operation",
 });
 }}
 >
 Warning
 </div>
</div>

```

```

 </Button>
 <Button
 variant="outline"
 onClick={() => {
 sonnerToast.loading("Loading", {
 description: "Please wait",
 });
 }}
 >
 Loading
 </Button>
 <Button
 variant="outline"
 onClick={() => {
 const promise = new Promise(resolve =>
 setTimeout(resolve, 2000)
);
 sonnerToast.promise(promise, {
 loading: "Processing...",
 success: "Processing complete!",
 error: "Processing failed",
 });
 }}
 >
 Promise
 </Button>
 </div>
 </div>
 </CardContent>
</Card>
</section>

{/* AI ChatBox Section */}
<section className="space-y-4">
 <h3 className="text-2xl font-semibold">AI ChatBox</h3>
 <Card>
 <CardContent className="pt-6">
 <div className="space-y-4">
 <div className="text-sm text-muted-foreground">
 <p>
 A ready-to-use chat interface component that integrates
 Features markdown rendering, auto-scrolling, and local
 </p>
 <p className="mt-2">

```

```

 This is a demo with simulated responses. In a real
 </p>
</div>
<AIChatBox
 messages={chatMessages}
 onSendMessage={handleChatSend}
 isLoading={isChatLoading}
 placeholder="Try sending a message..."
 height="500px"
 emptyStateMessage="How can I help you today?"
 suggestedPrompts=[
 "What is React?",
 "Explain TypeScript",
 "How to use tRPC?",
 "Best practices for web development",
]
 />
</div>
</CardContent>
</Card>
</section>
</div>
</main>

<footer className="border-t py-6 mt-12">
 <div className="container text-center text-sm text-muted-foreground">
 <p>Shadcn/ui Component Showcase</p>
 </div>
</footer>
</div>
);
}

```

## File: ./client/src/pages/Home.tsx

```
import { Button } from "@components/ui/button";
import { Mail, Github, Linkedin, Download, ExternalLink, ArrowRight } from "lucide-react";
import { useEffect, useState } from "react";
import { trpc } from "@lib/trpc";
import { toast } from "sonner";

/**
 * Portfolio Design Philosophy: Data-Driven Professional
 * - Dark theme with cyan (#00d9ff) and purple (#7c3aed) accents
 * - Metric-focused achievements and quantifiable impact
 * - Smooth scroll animations and interactions
 * - Professional yet approachable aesthetic
 */

interface NavItem {
 id: string;
 label: string;
 href: string;
}

const navItems: NavItem[] = [
 { id: "hero", label: "Home", href: "#hero" },
 { id: "about", label: "About", href: "#about" },
 { id: "skills", label: "Skills", href: "#skills" },
 { id: "experience", label: "Experience", href: "#experience" },
 { id: "contact", label: "Contact", href: "#contact" },
];

interface Skill {
 category: string;
 items: string[];
}

const skills: Skill[] = [
 {
 category: "Data Analytics & Technical",
 items: ["SQL", "Python (Pandas, NumPy)", "Data Wrangling", "ETL Automati"],
 },
 {
 category: "Analytics & Business",
 items: ["Data Quality Analysis", "Root-Cause Analysis", "Stakeholder"],
 },
];
```

```

 },
 {
 category: "Tools & Platforms",
 items: ["Snowflake", "Power BI", "Jira", "Confluence", "GraphQL"],
 },
],
];

interface Achievement {
 metric: string;
 description: string;
 icon: string;
}

const achievements: Achievement[] = [
 {
 metric: "70%",
 description: "Data product performance boost through ETL automation",
 icon: "📈",
 },
 {
 metric: "60%+",
 description: "Validation efficiency improvement, scaled from 1 to 100",
 icon: "✅",
 },
 {
 metric: "$96K",
 description: "Annual cost savings generated",
 icon: "💰",
 },
 {
 metric: "9%",
 description: "Reporting accuracy improvement through data analysis",
 icon: "🎯",
 },
];

interface ExperienceItem {
 title: string;
 company: string;
 period: string;
 highlights: string[];
}

const experience: ExperienceItem[] = [

```

```

{
 title: "Senior Product Analyst (Data Analytics Focus)",
 company: "FactSet Research Systems",
 period: "Jun 2022 - Jan 2026",
 highlights: [
 "Boosted data product performance by 70% through ETL-driven automation",
 "Performed data wrangling by extracting, cleaning, transforming and loading data",
 "Automated Python and SQL pipelines, reducing manual effort by 40%",
 "Defined and tracked success metrics and KPIs to measure data product performance",
 "Analyzed large-scale datasets using Snowflake and SQL to identify trends and anomalies",
 "Conducted data validation and QA analysis, improving validation efficiency",
 "Built Power BI dashboards and Excel reports to visualize KPIs, trends, and forecasts"
],
},
];

export default function Home() {
 const [activeSection, setActiveSection] = useState("hero");
 const [scrolled, setScrolled] = useState(false);
 const [formData, setFormData] = useState({ name: "", email: "", message: "" });
 const [isSubmitting, setIsSubmitting] = useState(false);

 useEffect(() => {
 const handleScroll = () => {
 setScrolled(window.scrollY > 50);

 // Update active section based on scroll position
 for (const item of navItems) {
 const element = document.getElementById(item.id);
 if (element) {
 const rect = element.getBoundingClientRect();
 if (rect.top ≤ 100 && rect.bottom ≥ 100) {
 setActiveSection(item.id);
 break;
 }
 }
 }
 };

 window.addEventListener("scroll", handleScroll);
 return () => window.removeEventListener("scroll", handleScroll);
 }, []);

 const scrollToSection = (id: string) => {

```

```

const element = document.getElementById(id);
if (element) {
 element.scrollToView({ behavior: "smooth" });
}
};

const sendMessageMutation = trpc.contact.sendMessage.useMutation();

const handleSubmit = async () => {
 if (!formData.name || !formData.email || !formData.message) {
 toast.error("Please fill in all fields");
 return;
 }

 setIsSubmitting(true);
 try {
 const result = await sendMessageMutation.mutateAsync(formData);
 if (result.success) {
 toast.success("Message sent successfully! We'll get back to you soon");
 setFormData({ name: "", email: "", message: "" });
 } else {
 toast.error(result.message || "Failed to send message");
 }
 } catch (error) {
 console.error("Error sending message:", error);
 toast.error("There was an error sending your message. Please try again");
 } finally {
 setIsSubmitting(false);
 }
};

return (
 <div className="min-h-screen bg-background text-foreground">
 {/* Navigation */}
 <nav
 className={`fixed top-0 left-0 right-0 z-50 transition-all duration-300
 scrolled ? "bg-background/80 backdrop-blur-md border-b border-border" : ""
 }`>
 >
 <div className="container flex items-center justify-between h-16">
 {/* Logo */}
 <div className="flex items-center gap-2">

 <span className="font-bold text-lg bg-gradient-to-r from-cyan
 Prashanth

 </div>

 {/* Nav Links */}
 <div className="hidden md:flex items-center gap-8">
 {navItems.map((item) => (
 <button
 key={item.id}
 onClick={() => scrollToSection(item.id)}
 className={`text-sm font-medium transition-colors duration
 activeSection === item.id ? "text-cyan-400" : "text-mut
 }`}
 >
 {item.label}
 </button>
))}
 </div>

 {/* CTA Button */}
 <Button
 size="sm"
 className="bg-cyan-500 hover:bg-cyan-600 text-background font
 onClick={() => scrollToSection("contact")}
 >
 Get in Touch
 </Button>
 </div>
</nav>

{/* Hero Section */}
<section
 id="hero"
 className="relative min-h-screen flex items-center justify-center
 >
 {/* Background Image */}
 <div
 className="absolute inset-0 z-0"
 style={{
 backgroundImage:

```



```

 size="lg"
 className="bg-cyan-500 hover:bg-cyan-600 text-background-
 onClick={() => scrollToSection("about")}
 >
 Explore My Work
 <ArrowRight className="ml-2 w-4 h-4 group-hover:translate
 </Button>
 <a
 href="/Prashanth_Terupelli_Resume.pdf"
 download
 className="inline-flex items-center justify-center px-6 p
 >
 <Download className="w-4 h-4 mr-2" />
 Download CV

</div>
</div>
</div>
</div>

{/* Scroll Indicator */}
<div className="absolute bottom-8 left-1/2 -translate-x-1/2 z-20
 <div className="w-6 h-10 border-2 border-cyan-400 rounded-full
 <div className="w-1 h-2 bg-cyan-400 rounded-full animate-puls
 </div>
 </div>
</section>

{/* About Section */}
<section id="about" className="py-20 bg-card/30">
 <div className="container max-w-4xl">
 <div className="space-y-12">
 <div className="text-center space-y-4">
 <h2 className="text-4xl md:text-5xl font-bold" style={{ fon
 <div className="w-16 h-1 bg-gradient-to-r from-cyan-400 to-
 </div>

 <div className="grid md:grid-cols-2 gap-12">
 <div className="space-y-6">
 <div>
 <h3 className="text-xl font-bold text-cyan-400 mb-3">Wh
 <p className="text-muted-foreground leading-relaxed">
 I'm an analytical and results-driven Senior Product Ar
 </p>
 </div>

```

```

<div>
 <h3 className="text-xl font-bold text-cyan-400 mb-3">My
 <p className="text-muted-foreground leading-relaxed">
 I believe in writing clean, maintainable code and fol
 </p>
</div>

<div>
 <h3 className="text-xl font-bold text-cyan-400 mb-3">My
 <p className="text-muted-foreground leading-relaxed">
 To continuously enhance my skills in data analytics,
 </p>
</div>
</div>

<div className="space-y-4">
 <div className="bg-background/50 border border-border rou
 <h4 className="font-bold text-cyan-400 mb-4">Quick Fact:
 <ul className="space-y-3 text-sm">
 <li className="flex items-start gap-3">
 >
 Location: Hyderabad, Telangan

 <li className="flex items-start gap-3">
 >
 Experience: 3.7+ years in Da

 <li className="flex items-start gap-3">
 >
 Education: B.Tech in Computer

 <li className="flex items-start gap-3">
 >
 Status: Immediate Joiner</sp

</div>

<div className="bg-background/50 border border-border rou
 <h4 className="font-bold text-cyan-400 mb-4">Connect Wi
 <div className="flex gap-3">
 <a
 href="mailto:tprashanth2301@gmail.com"

```



```

 className="bg-card/50 border border-border rounded-xl p-4"
 >
 <h3 className="font-bold text-lg mb-4 text-cyan-400 group-hover:underline">
 {skillGroup.category}
 </h3>
 <div className="flex flex-wrap gap-2">
 {skillGroup.items.map((skill, sIdx) => (
 <span
 key={sIdx}
 className="px-3 py-1 bg-background border border-border rounded"
 >
 {skill}

))}
 </div>
 </div>
 </div>
 </div>
</div>
</section>

{/* Experience Section */}
<section id="experience" className="py-20 bg-card/30">
 <div className="container max-w-4xl">
 <div className="space-y-12">
 <div className="text-center space-y-4">
 <h2 className="text-4xl md:text-5xl font-bold" style={{ font: 'serif' }}>
 Experience
 <div className="w-16 h-1 bg-gradient-to-r from-cyan-400 to-cyan-600" style={{ margin: '0 auto' }}>
 </div>
 </div>

 <div className="space-y-8">
 {experience.map((item, idx) => (
 <div
 key={idx}
 className="relative pl-8 border-l-2 border-border pb-8"
 >
 <div className="absolute -left-[9px] top-0 w-4 h-4 rounded-full bg-background border border-border" style={{ margin-left: -9px }}>
 <div className="bg-background/50 border border-border rounded" style={{ margin-left: 10px }}>
 <div className="flex flex-col md:flex-row md:items-center">
 <div>
 <h3 className="text-xl font-bold text-foreground">
 {item.title}
 <p className="text-cyan-400 font-medium">{item.company}
 </div>

```



```

<div className="flex items-center gap-4 group">
 <div className="p-3 bg-purple-500/10 border border-purple-500">
 <Link href="#" className="w-6 h-6 text-purple-400" />
 </div>
 <div>
 <p className="text-sm text-muted-foreground">Profession

 linkedin.com/in/prashanth

 </div>
</div>
</div>
</div>

<div className="bg-card/50 border border-border rounded-2xl p-4">
 <div className="space-y-4">
 <div className="space-y-2">
 <label className="text-sm font-medium">Full Name</label>
 <input
 type="text"
 placeholder="John Doe"
 className="w-full bg-background border border-border"
 value={formData.name}
 onChange={(e) => setFormData({ ...formData, name: e.target.value })}
 />
 </div>
 <div className="space-y-2">
 <label className="text-sm font-medium">Email Address</label>
 <input
 type="email"
 placeholder="john@example.com"
 className="w-full bg-background border border-border"
 value={formData.email}
 onChange={(e) => setFormData({ ...formData, email: e.target.value })}
 />
 </div>
 <div className="space-y-2">
 <label className="text-sm font-medium">Your Message</label>
 <textarea
 rows={4}
 placeholder="Tell me about your project..."
 className="w-full bg-background border border-border"
 value={formData.message}
 />
 </div>
 </div>
</div>

```



```

{ /* Footer */}
<footer className="border-t border-border py-8 bg-card/30">
 <div className="container text-center text-muted-foreground text-sm">
 <p>© 2026 Prashanth Terupelli. All rights reserved.</p>
 <p className="mt-2">Crafted with passion by Prashanth Terupelli.</p>
 </div>
</footer>

{ /* Scroll to Top Button */}
{scrolled && (
 <button
 onClick={() => scrollToSection("hero")}
 className="fixed bottom-8 right-8 z-40 p-3 bg-cyan-500 hover:bg-cyan-600"
 >
 <ArrowRight className="w-4 h-4 rotate-[-90deg]" />
 </button>
)}
</div>
);
}

```

## File: ./client/src/pages/NotFound.tsx

```
import { Button } from "@components/ui/button";
import { Card, CardContent } from "@components/ui/card";
import { AlertCircle, Home } from "lucide-react";
import { useLocation } from "wouter";

export default function NotFound() {
 const [, setLocation] = useLocation();

 const handleGoHome = () => {
 setLocation("/");
 };

 return (
 <div className="min-h-screen w-full flex items-center justify-center">
 <Card className="w-full max-w-lg mx-4 shadow-lg border-0 bg-white/80">
 <CardContent className="pt-8 pb-8 text-center">
 <div className="flex justify-center mb-6">
 <div className="relative">
 <div className="absolute inset-0 bg-red-100 rounded-full animate-pulse"></div>
 <AlertCircle className="relative h-16 w-16 text-red-500" />
 </div>
 </div>
 <h1 className="text-4xl font-bold text-slate-900 mb-2">404</h1>
 <h2 className="text-xl font-semibold text-slate-700 mb-4">
 Page Not Found
 </h2>
 <p className="text-slate-600 mb-8 leading-relaxed">
 Sorry, the page you are looking for doesn't exist.

 It may have been moved or deleted.
 </p>
 <div
 id="not-found-button-group"
 className="flex flex-col sm:flex-row gap-3 justify-center"
 >
 <Button
 onClick={handleGoHome}
 />
 </div>
 </CardContent>
 </Card>
 </div>
);
}
```

```

 className="bg-blue-600 hover:bg-blue-700 text-white px-6 py-4"
 >
 <Home className="w-4 h-4 mr-2" />
 Go Home
 </Button>
 </div>
 </CardContent>
</Card>
</div>
);
}

```

## File: ./components.json

```

{
 "$schema": "https://ui.shadcn.com/schema.json",
 "style": "new-york",
 "rsc": false,
 "tsx": true,
 "tailwind": {
 "css": "client/src/index.css",
 "baseColor": "neutral",
 "cssVariables": true,
 "prefix": ""
 },
 "aliases": {
 "components": "@components",
 "utils": "@lib/utils",
 "ui": "@components/ui",
 "lib": "@lib",
 "hooks": "@hooks"
 }
}

```

## File: ./drizzle.config.ts

```
import { defineConfig } from "drizzle-kit";

const connectionString = process.env.DATABASE_URL;
if (!connectionString) {
 throw new Error("DATABASE_URL is required to run drizzle commands");
}

export default defineConfig({
 schema: "./drizzle/schema.ts",
 out: "./drizzle",
 dialect: "mysql",
 dbCredentials: {
 url: connectionString,
 },
});
```

## File: ./drizzle/meta/0000\_snapshot.json

```
{
 "version": "5",
 "dialect": "mysql",
 "id": "55370a1e-9687-4f69-a89b-f505347dd046",
 "prevId": "000000000-0000-0000-0000-000000000000",
 "tables": {
 "users": {
 "name": "users",
 "columns": {
 "id": {
 "name": "id",
 "type": "int",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": true
 },
 "openId": {
 "name": "openId",
 "type": "varchar(64)",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false
 },
 "name": {
 "name": "name",
 "type": "text",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "email": {
 "name": "email",
 "type": "varchar(320)",
 "primaryKey": false,
 "notNull": false,
 "autoincrement": false
 },
 "loginMethod": {
 "name": "loginMethod",
 "type": "varchar(64)",
 "primaryKey": false,
```

```
 "notNull": false,
 "autoincrement": false
 },
 "role": {
 "name": "role",
 "type": "enum('user','admin')",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "'user'"
 },
 "createdAt": {
 "name": "createdAt",
 "type": "timestamp",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(now())"
 },
 "updatedAt": {
 "name": "updatedAt",
 "type": "timestamp",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "onUpdate": true,
 "default": "(now())"
 },
 "lastSignedIn": {
 "name": "lastSignedIn",
 "type": "timestamp",
 "primaryKey": false,
 "notNull": true,
 "autoincrement": false,
 "default": "(now())"
 }
},
"indexes": {},
"foreignKeys": {},
"compositePrimaryKeys": {
 "users_id": {
 "name": "users_id",
 "columns": [
 "id"
```



```
``typescript
import {} from "./schema";
```

## File: ./drizzle/schema.ts

```
import { int, mysqlEnum, mysqlTable, text, timestamp, varchar } from "drizzle-orm";

/**
 * Core user table backing auth flow.
 * Extend this file with additional tables as your product grows.
 * Columns use camelCase to match both database fields and generated types.
 */
export const users = mysqlTable("users", {
 /**
 * Surrogate primary key. Auto-incremented numeric value managed by the database.
 * Use this for relations between tables.
 */
 id: int("id").autoincrement().primaryKey(),
 /** Manus OAuth identifier (openId) returned from the OAuth callback. Unique per user. */
 openId: varchar("openId", { length: 64 }).notNull().unique(),
 name: text("name"),
 email: varchar("email", { length: 320 }),
 loginMethod: varchar("loginMethod", { length: 64 }),
 role: mysqlEnum("role", ["user", "admin"]).default("user").notNull(),
 createdAt: timestamp("createdAt").defaultNow().notNull(),
 updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
 lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
});

export type User = typeof users.$inferSelect;
export type InsertUser = typeof users.$inferInsert;

// TODO: Add your tables here

File: ./package.json
```typescript
{
  "name": "prashanth-portfolio",
  "version": "1.0.0",
  "type": "module",
  "license": "MIT",
  "scripts": {
    "dev": "NODE_ENV=development tsx watch server/_core/index.ts",
    "build": "vite build && esbuild server/_core/index.ts --platform=node",
    "start": "NODE_ENV=production node dist/index.js",
    "check": "tsc --noEmit",
  },
}
```

```
"format": "prettier --write .",
"test": "vitest run",
"db:push": "drizzle-kit generate && drizzle-kit migrate"
},
"dependencies": {
  "@aws-sdk/client-s3": "^3.693.0",
  "@aws-sdk/s3-request-presigner": "^3.693.0",
  "@hookform/resolvers": "^5.2.2",
  "@radix-ui/react-accordion": "^1.2.12",
  "@radix-ui/react-alert-dialog": "^1.1.15",
  "@radix-ui/react-aspect-ratio": "^1.1.7",
  "@radix-ui/react-avatar": "^1.1.10",
  "@radix-ui/react-checkbox": "^1.3.3",
  "@radix-ui/react-collapsible": "^1.1.12",
  "@radix-ui/react-context-menu": "^2.2.16",
  "@radix-ui/react-dialog": "^1.1.15",
  "@radix-ui/react-dropdown-menu": "^2.1.16",
  "@radix-ui/react-hover-card": "^1.1.15",
  "@radix-ui/react-label": "^2.1.7",
  "@radix-ui/react-menubar": "^1.1.16",
  "@radix-ui/react-navigation-menu": "^1.2.14",
  "@radix-ui/react-popover": "^1.1.15",
  "@radix-ui/react-progress": "^1.1.7",
  "@radix-ui/react-radio-group": "^1.3.8",
  "@radix-ui/react-scroll-area": "^1.2.10",
  "@radix-ui/react-select": "^2.2.6",
  "@radix-ui/react-separator": "^1.1.7",
  "@radix-ui/react-slider": "^1.3.6",
  "@radix-ui/react-slot": "^1.2.3",
  "@radix-ui/react-switch": "^1.2.6",
  "@radix-ui/react-tabs": "^1.1.13",
  "@radix-ui/react-toggle": "^1.1.10",
  "@radix-ui/react-toggle-group": "^1.1.11",
  "@radix-ui/react-tooltip": "^1.2.8",
  "@tanstack/react-query": "^5.90.2",
  "@trpc/client": "^11.6.0",
  "@trpc/react-query": "^11.6.0",
  "@trpc/server": "^11.6.0",
  "axios": "^1.12.0",
  "class-variance-authority": "^0.7.1",
  "clsx": "^2.1.1",
  "cmdk": "^1.1.1",
  "cookie": "^1.0.2",
  "date-fns": "^4.1.0",
```

```
"dotenv": "^17.2.2",
"drizzle-orm": "^0.44.5",
"embla-carousel-react": "^8.6.0",
"express": "^4.21.2",
"framer-motion": "^12.23.22",
"input-otp": "^1.4.2",
"jose": "6.1.0",
"lucide-react": "^0.453.0",
"mysql2": "^3.15.0",
"nanoid": "^5.1.5",
"next-themes": "^0.4.6",
"nodemailer": "^9.0.1",
"react": "^19.2.1",
"react-day-picker": "^9.11.1",
"react-dom": "^19.2.1",
"react-hook-form": "^7.64.0",
"react-resizable-panels": "^3.0.6",
"recharts": "^2.15.2",
"sonner": "^2.0.7",
"streamdown": "^1.4.0",
"superjson": "^1.13.3",
"tailwind-merge": "^3.3.1",
"tailwindcss-animate": "^1.0.7",
"vaul": "^1.1.2",
"wouter": "^3.3.5",
"zod": "^4.1.12"
},
"devDependencies": {
  "@builder.io/vite-plugin-jsx-loc": "^0.1.1",
  "@tailwindcss/typography": "^0.5.15",
  "@tailwindcss/vite": "^4.1.3",
  "@types/express": "4.17.21",
  "@types/google.maps": "^3.58.1",
  "@types/node": "^24.7.0",
  "@types/nodemailer": "^8.0.1",
  "@types/react": "^19.2.1",
  "@types/react-dom": "^19.2.1",
  "@vitejs/plugin-react": "^5.0.4",
  "add": "^2.0.6",
  "autoprefixer": "^10.4.20",
  "drizzle-kit": "^0.31.4",
  "esbuild": "^0.25.0",
  "pnpm": "^10.15.1",
  "postcss": "^8.4.47",
```

```
"prettier": "^3.6.2",
"tailwindcss": "^4.1.14",
"tsx": "^4.19.1",
"tw-animate-css": "^1.4.0",
"typescript": "5.9.3",
"vite": "^7.1.7",
"vite-plugin-manus-runtime": "^0.0.58",
"vitest": "^2.1.4"
},
"packageManager": "pnpm@10.4.1+sha512.c753b6c3ad7afa13af388fa6d808035a0",
"pnpm": {
  "patchedDependencies": {
    "wouter@3.7.1": "patches/wouter@3.7.1.patch"
  },
  "overrides": {
    "tailwindcss>nanoid": "3.3.7"
  }
}
}
```

File: ./server/_core/context.ts

```
import type { CreateExpressContextOptions } from "@trpc/server/adapters/express";
import type { User } from "../../drizzle/schema";
import { sdk } from "../sdk";

export type TrpcContext = {
  req: CreateExpressContextOptions["req"];
  res: CreateExpressContextOptions["res"];
  user: User | null;
};

export async function createContext(
  opts: CreateExpressContextOptions
): Promise<TrpcContext> {
  let user: User | null = null;

  try {
    user = await sdk.authenticateRequest(opts.req);
  } catch (error) {
    // Authentication is optional for public procedures.
    user = null;
  }

  return {
    req: opts.req,
    res: opts.res,
    user,
  };
}
```

File: ./server/_core/cookies.ts

```
import type { CookieOptions, Request } from "express";

const LOCAL_HOSTS = new Set(["localhost", "127.0.0.1", "::1"]);

function isIpAddress(host: string) {
  // Basic IPv4 check and IPv6 presence detection.
  if (/^\d{1,3}(\.\d{1,3}){3}$/.test(host)) return true;
  return host.includes(":");
}

function isSecureRequest(req: Request) {
  if (req.protocol === "https") return true;

  const forwardedProto = req.headers["x-forwarded-proto"];
  if (!forwardedProto) return false;

  const protoList = Array.isArray(forwardedProto)
    ? forwardedProto
    : forwardedProto.split(",");

  return protoList.some(proto => proto.trim().toLowerCase() === "https");
}

export function getSessionCookieOptions(
  req: Request
): Pick<CookieOptions, "domain" | "httpOnly" | "path" | "sameSite" | "secure"> {
  // const hostname = req.hostname;
  // const shouldSetDomain =
  //   hostname &&
  //   !LOCAL_HOSTS.has(hostname) &&
  //   !isIpAddress(hostname) &&
  //   hostname !== "127.0.0.1" &&
  //   hostname !== "::1";

  // const domain =
  //   shouldSetDomain && !hostname.startsWith(".")
  //     ? `.${hostname}`
  //     : shouldSetDomain
  //       ? hostname
  //       : undefined;
}
```

```
return {  
  httpOnly: true,  
  path: "/",  
  sameSite: "none",  
  secure: isSecureRequest(req),  
};  
}
```

File: ./server/_core/dataApi.ts

```
/**
 * Quick example (matches curl usage):
 *   await callDataApi("Youtube/search", {
 *     query: { gl: "US", hl: "en", q: "manus" },
 *   })
 */
import { ENV } from "../env";

export type DataApiCallOptions = {
  query?: Record<string, unknown>;
  body?: Record<string, unknown>;
  pathParams?: Record<string, unknown>;
  formData?: Record<string, unknown>;
};

export async function callDataApi(
  apiId: string,
  options: DataApiCallOptions = {}
): Promise<unknown> {
  if (!ENV.forgeApiUrl) {
    throw new Error("BUILT_IN_FORGE_API_URL is not configured");
  }
  if (!ENV.forgeApiKey) {
    throw new Error("BUILT_IN_FORGE_API_KEY is not configured");
  }

  // Build the full URL by appending the service path to the base URL
  const baseUrl = ENV.forgeApiUrl.endsWith("/") ? ENV.forgeApiUrl : `${ENV.forgeApiUrl}/`;
  const fullUrl = new URL("webdevtoken.v1.WebDevService/CallApi", baseUrl);

  const response = await fetch(fullUrl, {
    method: "POST",
    headers: {
      accept: "application/json",
      "content-type": "application/json",
      "connect-protocol-version": "1",
      authorization: `Bearer ${ENV.forgeApiKey}`,
    },
    body: JSON.stringify({
      apiId,
      query: options.query,
    })
  });
}
```

```

        body: options.body,
        path_params: options.pathParams,
        multipart_form_data: options.formData,
    )),
});

if (!response.ok) {
    const detail = await response.text().catch(() => "");
    throw new Error(
        `Data API request failed (${response.status} ${response.statusText})`
    );
}

const payload = await response.json().catch(() => ({}));
if (payload && typeof payload === "object" && "jsonData" in payload) {
    try {
        return JSON.parse((payload as Record<string, string>).jsonData ?? "");
    } catch {
        return (payload as Record<string, unknown>).jsonData;
    }
}
return payload;
}

```

File: ./server/_core/env.ts

```

export const ENV = {
    appId: process.env.VITE_APP_ID ?? "",
    cookieSecret: process.env.JWT_SECRET ?? "",
    databaseUrl: process.env.DATABASE_URL ?? "",
    oauthServerUrl: process.env.OAUTH_SERVER_URL ?? "",
    ownerOpenId: process.env.OWNER_OPEN_ID ?? "",
    isProduction: process.env.NODE_ENV === "production",
    forgeApiUrl: process.env.BUILT_IN_FORGE_API_URL ?? "",
    forgeApiKey: process.env.BUILT_IN_FORGE_API_KEY ?? "",
    gmailUser: process.env.GMAIL_USER ?? "",
    gmailAppPassword: process.env.GMAIL_APP_PASSWORD ?? "",
};

```

File: ./server/_core/heartbeat.ts

```
import { TRPCError } from "@trpc/server";
import { ENV } from "../env";

export type HeartbeatJob = {
  name: string;
  /**
   * 6-field cron with seconds (`sec min hour dom mon dow`), UTC, min interval
   * Use `0` for the seconds field – e.g. `"0 0 9 * * *"` is daily 09:00 UTC
   * See periodic-updates.md.
   */
  cron: string;
  /** Callback path. MUST start with `/api/scheduled/`. */
  path: string;
  method?: "POST" | "PUT";
  payload?: unknown;
  description?: string;
};

/**
 * Update patch. All fields optional; unset = leave unchanged.
 * `enable`: true = resume, false = pause; omit = unchanged.
 * `name` is the (project, owner)-scope key and cannot be changed.
 */
export type HeartbeatJobUpdate = Partial<Omit<HeartbeatJob, "name">> & {
  enable?: boolean;
};

export type HeartbeatJobInfo = {
  taskId: string;
  name: string;
  userId: string;
  description: string;
  cronExpression: string;
  callbackPath: string;
  callbackMethod: string;
  callbackPayload: string;
  isEnabled: boolean;
  createdAt?: string | null;
  lastExecutedAt?: string | null;
  nextExecutionAt?: string | null;
};
```

```

const SERVICE = "webdevtoken.v1.WebDevService";

const buildEndpoint = (rpc: string): string => {
  if (!ENV.forgeApiUrl) {
    throw new TRPCError({
      code: "INTERNAL_SERVER_ERROR",
      message: "Heartbeat service URL is not configured (BUILT_IN_FORGE_API_URL)",
    });
  }
  if (!ENV.forgeApiKey) {
    throw new TRPCError({
      code: "INTERNAL_SERVER_ERROR",
      message: "Heartbeat service API key is not configured (BUILT_IN_FORGE_API_KEY)",
    });
  }
  const baseUrl = ENV.forgeApiUrl;
  const normalizedBase = baseUrl.endsWith("/") ? baseUrl : `${baseUrl}/`;
  return new URL(`${SERVICE}/${rpc}`, normalizedBase).toString();
};

const callForge = async <T>({
  rpc: string,
  body: Record<string, unknown>,
  userSession: string
}): Promise<T> => {
  const endpoint = buildEndpoint(rpc);
  const headers: Record<string, string> = {
    accept: "application/json",
    authorization: `Bearer ${ENV.forgeApiKey}`,
    "content-type": "application/json",
    "connect-protocol-version": "1",
  };
  // userSession is the decoded `app_session_id` cookie value (NOT the raw cookie value)
  // Cookie header). Empty string falls back to the project owner identity
  if (userSession) {
    headers["x-manus-user-session"] = userSession;
  }

  let response: Response;
  try {
    response = await fetch(endpoint, {
      method: "POST",
      headers,
    });
  } catch (error) {
    console.error("Heartbeat service call failed", error);
    throw new TRPCError({
      code: "INTERNAL_SERVER_ERROR",
      message: "Heartbeat service call failed",
    });
  }
  if (!response.ok) {
    throw new TRPCError({
      code: "INTERNAL_SERVER_ERROR",
      message: `Heartbeat service call failed with status ${response.status}`,
    });
  }
  return response.json();
};

```

```

        body: JSON.stringify(body),
    });
} catch (error) {
    throw new TRPCError({
        code: "INTERNAL_SERVER_ERROR",
        message: `Heartbeat ${rpc} network error: ${String(error)}`,
    });
}

if (!response.ok) {
    const detail = await response.text().catch(() => "");
    throw mapForgeError(response, detail, rpc);
}
return (await response.json()) as T;
};

const mapForgeError = (
    response: Response,
    detail: string,
    rpc: string
): TRPCError => {
    const status = response.status;
    let code: TRPCError["code"] = "INTERNAL_SERVER_ERROR";
    if (status === 401) code = "UNAUTHORIZED";
    else if (status === 403) code = "FORBIDDEN";
    else if (status === 404) code = "NOT_FOUND";
    else if (status === 400 || status === 422) code = "BAD_REQUEST";
    else if (status === 409) code = "CONFLICT";
    else if (status === 429) code = "TOO_MANY_REQUESTS";
    return new TRPCError({
        code,
        message: `Heartbeat ${rpc} failed (${status})${detail ? `: ${detail}` : ""}`,
    });
};

const stringifyPayload = (payload: unknown): string => {
    if (payload === undefined || payload === null) return "{}";
    if (typeof payload === "string") return payload;
    return JSON.stringify(payload);
};

const validateCallbackPath = (path: string): void => {
    if (!path || !path.startsWith("/api/scheduled/")) {
        throw new TRPCError({

```

```

        code: "BAD_REQUEST",
        message: "callback path must start with /api/scheduled/",
    });
}
};

/**
 * Create a new HTTP cron job. Returns the assigned `taskId` to persist
 * your business row so callbacks can dereference it.
 */
export async function createHeartbeatJob(
    job: HeartbeatJob,
    userSession: string
): Promise<{ taskId: string; nextExecutionAt?: string | null }> {
    validateCallbackPath(job.path);
    return callForge<{ taskId: string; nextExecutionAt?: string | null }>(
        "CreateHeartbeatJob",
        {
            name: job.name,
            cronExpression: job.cron,
            callbackPath: job.path,
            callbackMethod: job.method ?? "POST",
            callbackPayload: stringifyPayload(job.payload),
            description: job.description ?? "",
        },
        userSession
    );
}

/**
 * Update an existing cron located by `taskId`. Only fields you pass in
 * `patch` are mutated. `enable` flips resume/pause; omit to leave alone.
 */
export async function updateHeartbeatJob(
    taskId: string,
    patch: HeartbeatJobUpdate,
    userSession: string
): Promise<{ nextExecutionAt?: string | null }> {
    if (patch.path !== undefined) validateCallbackPath(patch.path);
    const body: Record<string, unknown> = { taskId };
    if (patch.cron !== undefined) body.cronExpression = patch.cron;
    if (patch.path !== undefined) body.callbackPath = patch.path;
    if (patch.method !== undefined) body.callbackMethod = patch.method;
    if (patch.payload !== undefined) {

```

```

    body.callbackPayload = stringifyPayload(patch.payload);
  }
  if (patch.description !== undefined) body.description = patch.description;
  if (patch.enable !== undefined) body.enable = patch.enable;
  return callForge<{ nextExecutionAt?: string | null }>(
    "UpdateHeartbeatJob",
    body,
    userSession
  );
}

/** Delete a cron located by `taskUid`. Idempotent on caller side. */
export async function deleteHeartbeatJob(
  taskUid: string,
  userSession: string
): Promise<void> {
  await callForge("DeleteHeartbeatJob", { taskUid }, userSession);
}

/**
 * List cron jobs owned by the resolved actor (end-user when `userSession`
 * is set, project owner otherwise) within the current project.
 *
 * `actorUserId` in the response echoes whose cron list you got back. End
 * user cannot list other users' crons via this SDK; cross-user inspection is
 * owner-only via the sandbox CLI (`manus-heartbeat list --user-id <uid>`).
 */
export async function listHeartbeatJobs(
  userSession: string,
  pagination?: { page?: number; pageSize?: number }
): Promise<{ total: number; actorUserId: string; jobs: HeartbeatJobInfo[] > {
  const body: Record<string, unknown> = {};
  if (pagination?.page !== undefined) body.page = pagination.page;
  if (pagination?.pageSize !== undefined) body.pageSize = pagination.pageSize;
  return callForge<{
    total: number;
    actorUserId: string;
    jobs: HeartbeatJobInfo[];
  }>("ListHeartbeatJobs", body, userSession);
}

```

File: ./server/_core/imageGeneration.ts

```
/**
 * Image generation helper using internal ImageService
 *
 * Example usage:
 *   const { url: imageUrl } = await generateImage({
 *     prompt: "A serene landscape with mountains"
 *   });
 *
 * For editing:
 *   const { url: imageUrl } = await generateImage({
 *     prompt: "Add a rainbow to this landscape",
 *     originalImages: [{
 *       url: "https://example.com/original.jpg",
 *       mimeType: "image/jpeg"
 *     }]
 *   });
 */
import { storagePut } from "server/storage";
import { ENV } from "../env";

export type GenerateImageOptions = {
  prompt: string;
  originalImages?: Array<{
    url?: string;
    b64Json?: string;
    mimeType?: string;
  }>;
};

export type GenerateImageResponse = {
  url?: string;
};

export async function generateImage(
  options: GenerateImageOptions
): Promise<GenerateImageResponse> {
  if (!ENV.forgeApiUrl) {
    throw new Error("BUILT_IN_FORGE_API_URL is not configured");
  }
  if (!ENV.forgeApiKey) {
    throw new Error("BUILT_IN_FORGE_API_KEY is not configured");
  }
}
```

```

}

// Build the full URL by appending the service path to the base URL
const baseUrl = ENV.forgeApiUrl.endsWith("/")
  ? ENV.forgeApiUrl
  : `${ENV.forgeApiUrl}/`;
const fullUrl = new URL(
  "images.v1.ImageService/GenerateImage",
  baseUrl
).toString();

const response = await fetch(fullUrl, {
  method: "POST",
  headers: {
    accept: "application/json",
    "content-type": "application/json",
    "connect-protocol-version": "1",
    authorization: `Bearer ${ENV.forgeApiKey}`,
  },
  body: JSON.stringify({
    prompt: options.prompt,
    original_images: options.originalImages || [],
  }),
});

if (!response.ok) {
  const detail = await response.text().catch(() => "");
  throw new Error(
    `Image generation request failed (${response.status} ${response.statusText})`
  );
}

const result = (await response.json()) as {
  image: {
    b64Json: string;
    mimeType: string;
  };
};

const base64Data = result.image.b64Json;
const buffer = Buffer.from(base64Data, "base64");

// Save to S3
const { url } = await storagePut(
  `generated/${Date.now()}.png`,

```

```
        buffer,  
        result.image.mimeType  
    );  
    return {  
        url,  
    };  
}
```

File: ./server/_core/index.ts

```
import "dotenv/config";
import express from "express";
import { createServer } from "http";
import net from "net";
import { createExpressMiddleware } from "@trpc/server/adapters/express";
import { registerOAuthRoutes } from "../oauth";
import { registerStorageProxy } from "../storageProxy";
import { appRouter } from "../routers";
import { createContext } from "../context";
import { serveStatic, setupVite } from "../vite";

function isPortAvailable(port: number): Promise<boolean> {
  return new Promise(resolve => {
    const server = net.createServer();
    server.listen(port, () => {
      server.close(() => resolve(true));
    });
    server.on("error", () => resolve(false));
  });
}

async function findAvailablePort(startPort: number = 3000): Promise<number> {
  for (let port = startPort; port < startPort + 20; port++) {
    if (await isPortAvailable(port)) {
      return port;
    }
  }
  throw new Error(`No available port found starting from ${startPort}`);
}

async function startServer() {
  const app = express();
  const server = createServer(app);
  // Configure body parser with larger size limit for file uploads
  app.use(express.json({ limit: "50mb" }));
  app.use(express.urlencoded({ limit: "50mb", extended: true }));
  registerStorageProxy(app);
  registerOAuthRoutes(app);
  // tRPC API
  app.use(
    "/api/trpc",
```

```
    createExpressMiddleware({
      router: appRouter,
      createContext,
    })
  );
  // development mode uses Vite, production mode uses static files
  if (process.env.NODE_ENV === "development") {
    await setupVite(app, server);
  } else {
    serveStatic(app);
  }

  const preferredPort = parseInt(process.env.PORT || "3000");
  const port = await findAvailablePort(preferredPort);

  if (port !== preferredPort) {
    console.log(`Port ${preferredPort} is busy, using port ${port} instead`);
  }

  server.listen(port, () => {
    console.log(`Server running on http://localhost:${port}/`);
  });
}

startServer().catch(console.error);
```

File: ./server/_core/llm.ts

```
import { ENV } from "./env";

export type Role = "system" | "user" | "assistant" | "tool" | "function";

export type TextContent = {
  type: "text";
  text: string;
};

export type ImageContent = {
  type: "image_url";
  image_url: {
    url: string;
    detail?: "auto" | "low" | "high";
  };
};

export type FileContent = {
  type: "file_url";
  file_url: {
    url: string;
    mime_type?: "audio/mpeg" | "audio/wav" | "application/pdf" | "audio/m
  };
};

export type MessageContent = string | TextContent | ImageContent | FileCo

export type Message = {
  role: Role;
  content: MessageContent | MessageContent[];
  name?: string;
  tool_call_id?: string;
};

export type Tool = {
  type: "function";
  function: {
    name: string;
    description?: string;
    parameters?: Record<string, unknown>;
  };
};
```

```

};

export type ToolChoicePrimitive = "none" | "auto" | "required";
export type ToolChoiceByName = { name: string };
export type ToolChoiceExplicit = {
  type: "function";
  function: {
    name: string;
  };
};

export type ToolChoice =
  | ToolChoicePrimitive
  | ToolChoiceByName
  | ToolChoiceExplicit;

export type InvokeParams = {
  messages: Message[];
  tools?: Tool[];
  toolChoice?: ToolChoice;
  tool_choice?: ToolChoice;
  maxTokens?: number;
  max_tokens?: number;
  outputSchema?: OutputSchema;
  output_schema?: OutputSchema;
  responseFormat?: ResponseFormat;
  response_format?: ResponseFormat;
  model?: string;
  thinking?: Record<string, unknown>;
  reasoning?: Record<string, unknown>;
};

export type ToolCall = {
  id: string;
  type: "function";
  function: {
    name: string;
    arguments: string;
  };
};

export type InvokeResult = {
  id: string;
  created: number;

```

```

model: string;
choices: Array<{
  index: number;
  message: {
    role: Role;
    content: string | Array<TextContent | ImageContent | FileContent>;
    tool_calls?: ToolCall[];
  };
  finish_reason: string | null;
}>;
usage?: {
  prompt_tokens: number;
  completion_tokens: number;
  total_tokens: number;
};
};

export type JsonSchema = {
  name: string;
  schema: Record<string, unknown>;
  strict?: boolean;
};

export type OutputSchema = JsonSchema;

export type ResponseFormat =
  | { type: "text" }
  | { type: "json_object" }
  | { type: "json_schema"; json_schema: JsonSchema };

const ensureArray = (
  value: MessageContent | MessageContent[]
): MessageContent[] => (Array.isArray(value) ? value : [value]);

const normalizeContentPart = (
  part: MessageContent
): TextContent | ImageContent | FileContent => {
  if (typeof part === "string") {
    return { type: "text", text: part };
  }

  if (part.type === "text") {
    return part;
  }

```

```

    if (part.type === "image_url") {
      return part;
    }

    if (part.type === "file_url") {
      return part;
    }

    throw new Error("Unsupported message content part");
  };

const normalizeMessage = (message: Message) => {
  const { role, name, tool_call_id } = message;

  if (role === "tool" || role === "function") {
    const content = ensureArray(message.content)
      .map(part => (typeof part === "string" ? part : JSON.stringify(part))
      .join("\n");

    return {
      role,
      name,
      tool_call_id,
      content,
    };
  }

  const contentParts = ensureArray(message.content).map(normalizeContentPart);

  // If there's only text content, collapse to a single string for compatibility
  if (contentParts.length === 1 && contentParts[0].type === "text") {
    return {
      role,
      name,
      content: contentParts[0].text,
    };
  }

  return {
    role,
    name,
    content: contentParts,
  };
};

```

```

};

const normalizeToolChoice = (
  toolChoice: ToolChoice | undefined,
  tools: Tool[] | undefined
): "none" | "auto" | ToolChoiceExplicit | undefined => {
  if (!toolChoice) return undefined;

  if (toolChoice === "none" || toolChoice === "auto") {
    return toolChoice;
  }

  if (toolChoice === "required") {
    if (!tools || tools.length === 0) {
      throw new Error(
        "tool_choice 'required' was provided but no tools were configured"
      );
    }

    if (tools.length > 1) {
      throw new Error(
        "tool_choice 'required' needs a single tool or specify the tool name"
      );
    }

    return {
      type: "function",
      function: { name: tools[0].function.name },
    };
  }

  if ("name" in toolChoice) {
    return {
      type: "function",
      function: { name: toolChoice.name },
    };
  }

  return toolChoice;
};

const resolveApiUrl = () =>
  ENV.forgeApiUrl && ENV.forgeApiUrl.trim().length > 0
    ? `${ENV.forgeApiUrl.replace(/\/$/, "")}/v1/chat/completions`

```

```

    : "https://forge.manus.im/v1/chat/completions";

const assertApiKey = () => {
  if (!ENV.forgeApiKey) {
    throw new Error("OPENAI_API_KEY is not configured");
  }
};

const normalizeResponseFormat = ({
  responseFormat,
  response_format,
  outputSchema,
  output_schema,
}): {
  responseFormat?: ResponseFormat;
  response_format?: ResponseFormat;
  outputSchema?: OutputSchema;
  output_schema?: OutputSchema;
}): {
  | { type: "json_schema"; json_schema: JsonSchema }
  | { type: "text" }
  | { type: "json_object" }
  | undefined => {
    const explicitFormat = responseFormat || response_format;
    if (explicitFormat) {
      if (
        explicitFormat.type === "json_schema" &&
        !explicitFormat.json_schema?.schema
      ) {
        throw new Error(
          "responseFormat json_schema requires a defined schema object"
        );
      }
    }
    return explicitFormat;
  }

  const schema = outputSchema || output_schema;
  if (!schema) return undefined;

  if (!schema.name || !schema.schema) {
    throw new Error("outputSchema requires both name and schema");
  }

  return {

```

```

    type: "json_schema",
    json_schema: {
      name: schema.name,
      schema: schema.schema,
      ...(typeof schema.strict === "boolean" ? { strict: schema.strict }
    },
  };
};

const RETRY_MAX_RETRIES = 4;
const RETRY_BASE_DELAY_MS = 500;
const RETRY_MAX_DELAY_MS = 30_000;

type FetchInit = NonNullable<Parameters<typeof fetch>[1]>;

const sleep = (ms: number) =>
  new Promise<void>(resolve => setTimeout(resolve, ms));

const parseRetryAfter = (value: string | null): number | undefined => {
  if (!value) return undefined;
  const seconds = Number(value);
  if (Number.isFinite(seconds)) return Math.max(0, seconds * 1000);
  const at = Date.parse(value);
  return Number.isNaN(at) ? undefined : Math.max(0, at - Date.now());
};

// Equal-jitter exponential backoff. The cap/2 floor guarantees a minimum
// delay so a misbehaving caller loop slows down instead of hammering the
// upstream while it keeps returning errors.
const computeBackoffDelay = (
  attempt: number,
  retryAfterMs?: number
): number => {
  const cap = Math.min(RETRY_BASE_DELAY_MS * 2 ** attempt, RETRY_MAX_DELAY_MS);
  const jittered = cap / 2 + Math.random() * (cap / 2);
  return Math.min(Math.max(jittered, retryAfterMs ?? 0), RETRY_MAX_DELAY_MS);
};

// Retries non-2xx responses and network errors with exponential backoff,
// returns the final Response so callers keep their existing error handling
const fetchWithBackoff = async (
  url: string,
  init: FetchInit
): Promise<Response> => {

```

```

let lastError: unknown;

for (let attempt = 0; attempt ≤ RETRY_MAX_RETRIES; attempt++) {
  try {
    const response = await fetch(url, init);
    if (response.ok || attempt === RETRY_MAX_RETRIES) {
      return response;
    }

    const retryAfterMs = parseRetryAfter(
      response.headers.get("retry-after")
    );
    try {
      await response.body?.cancel();
    } catch {
      // Body already settled; nothing to clean up.
    }
    console.warn(
      `LLM request retry ${attempt + 1}/${RETRY_MAX_RETRIES} after status`
    );
    await sleep(computeBackoffDelay(attempt, retryAfterMs));
  } catch (error) {
    lastError = error;
    if (attempt === RETRY_MAX_RETRIES) throw error;
    console.warn(
      `LLM request retry ${attempt + 1}/${RETRY_MAX_RETRIES} after network`
    );
    await sleep(computeBackoffDelay(attempt));
  }
}

throw lastError instanceof Error
  ? lastError
  : new Error("LLM request failed after exhausting retries");
};

export async function invokeLLM(params: InvokeParams): Promise<InvokeResult> {
  assertApiKey();

  const {
    messages,
    tools,
    toolChoice,
    tool_choice,
  } = params;

```

```
outputSchema,
output_schema,
responseFormat,
response_format,
model,
thinking,
reasoning,
maxTokens,
max_tokens,
} = params;

const payload: Record<string, unknown> = {
  messages: messages.map(normalizeMessage),
};

if (model) {
  payload.model = model;
}

if (tools && tools.length > 0) {
  payload.tools = tools;
}

const normalizedToolChoice = normalizeToolChoice(
  toolChoice || tool_choice,
  tools
);
if (normalizedToolChoice) {
  payload.tool_choice = normalizedToolChoice;
}

const resolvedMaxTokens = max_tokens ?? maxTokens;
if (typeof resolvedMaxTokens === "number") {
  payload.max_tokens = resolvedMaxTokens;
}

if (thinking) {
  payload.thinking = thinking;
}
if (reasoning) {
  payload.reasoning = reasoning;
}

const normalizedResponseFormat = normalizeResponseFormat({
```

```

    responseFormat,
    response_format,
    outputSchema,
    output_schema,
  });

  if (normalizedResponseFormat) {
    payload.response_format = normalizedResponseFormat;
  }

  const response = await fetchWithBackoff(resolveApiUrl(), {
    method: "POST",
    headers: {
      "content-type": "application/json",
      authorization: `Bearer ${ENV.forgeApiKey}`,
    },
    body: JSON.stringify(payload),
  });

  if (!response.ok) {
    const errorText = await response.text();
    throw new Error(
      `LLM invoke failed: ${response.status} ${response.statusText} - ${e}
    );
  }

  return (await response.json()) as InvokeResult;
}

export type ModelInfo = {
  id: string;
  object: string;
  created: number;
  owned_by: string;
};

export type ModelsResponse = {
  object: string;
  data: ModelInfo[];
};

export async function listLLMModels(): Promise<ModelsResponse> {
  assertApiKey();

```

```
const url = ENV.forgeApiUrl && ENV.forgeApiUrl.trim().length > 0
  ? `${ENV.forgeApiUrl.replace(/\$/ , "")}/v1/models`
  : "https://forge.manus.im/v1/models";

const response = await fetchWithBackoff(url, {
  headers: { authorization: `Bearer ${ENV.forgeApiKey}` },
});

if (!response.ok) {
  const errorText = await response.text();
  throw new Error(
    `List LLM models failed: ${response.status} ${response.statusText} .
  );
}

return (await response.json()) as ModelsResponse;
}
```

File: ./server/_core/map.ts

```
/**
 * Google Maps API Integration for Manus WebDev Templates
 *
 * Main function: makeRequest<T>(endpoint, params) - Makes authenticated requests
 * All credentials are automatically injected. Array parameters use | as separator
 *
 * See API examples below the type definitions for usage patterns.
 */

import { ENV } from "../env";

// =====
// Configuration
// =====

type MapsConfig = {
  baseUrl: string;
  apiKey: string;
};

function getMapsConfig(): MapsConfig {
  const baseUrl = ENV.forgeApiUrl;
  const apiKey = ENV.forgeApiKey;

  if (!baseUrl || !apiKey) {
    throw new Error(
      "Google Maps proxy credentials missing: set BUILT_IN_FORGE_API_URL & API_KEY"
    );
  }

  return {
    baseUrl: baseUrl.replace(/\/+$/, ""),
    apiKey,
  };
}

// =====
// Core Request Handler
// =====

interface RequestOptions {
```

```

    method?: "GET" | "POST";
    body?: Record<string, unknown>;
}

/**
 * Make authenticated requests to Google Maps APIs
 *
 * @param endpoint - The API endpoint (e.g., "/maps/api/geocode/json")
 * @param params - Query parameters for the request
 * @param options - Additional request options
 * @returns The API response
 */
export async function makeRequest<T = unknown>(
    endpoint: string,
    params: Record<string, unknown> = {},
    options: RequestOptions = {}
): Promise<T> {
    const { baseUrl, apiKey } = getMapsConfig();

    // Construct full URL: baseUrl + /v1/maps/proxy + endpoint
    const url = new URL(`${baseUrl}/v1/maps/proxy${endpoint}`);

    // Add API key as query parameter (standard Google Maps API authentication)
    url.searchParams.append("key", apiKey);

    // Add other query parameters
    Object.entries(params).forEach(([key, value]) => {
        if (value !== undefined && value !== null) {
            url.searchParams.append(key, String(value));
        }
    });

    const response = await fetch(url.toString(), {
        method: options.method || "GET",
        headers: {
            "Content-Type": "application/json",
        },
        body: options.body ? JSON.stringify(options.body) : undefined,
    });

    if (!response.ok) {
        const errorText = await response.text();
        throw new Error(
            `Google Maps API request failed (${response.status} ${response.statusText})`
        );
    }
}

```

```

    );
}

return (await response.json()) as T;
}

// =====
// Type Definitions
// =====

export type TravelMode = "driving" | "walking" | "bicycling" | "transit";
export type MapType = "roadmap" | "satellite" | "terrain" | "hybrid";
export type SpeedUnit = "KPH" | "MPH";

export type LatLng = {
  lat: number;
  lng: number;
};

export type DirectionsResult = {
  routes: Array<{
    legs: Array<{
      distance: { text: string; value: number };
      duration: { text: string; value: number };
      start_address: string;
      end_address: string;
      start_location: LatLng;
      end_location: LatLng;
      steps: Array<{
        distance: { text: string; value: number };
        duration: { text: string; value: number };
        html_instructions: string;
        travel_mode: string;
        start_location: LatLng;
        end_location: LatLng;
      }>;
    }>;
    overview_polyline: { points: string };
    summary: string;
    warnings: string[];
    waypoint_order: number[];
  }>;
  status: string;
};

```

```
export type DistanceMatrixResult = {
  rows: Array<{
    elements: Array<{
      distance: { text: string; value: number };
      duration: { text: string; value: number };
      status: string;
    }>;
  }>;
  origin_addresses: string[];
  destination_addresses: string[];
  status: string;
};
```

```
export type GeocodingResult = {
  results: Array<{
    address_components: Array<{
      long_name: string;
      short_name: string;
      types: string[];
    }>;
    formatted_address: string;
    geometry: {
      location: LatLng;
      location_type: string;
      viewport: {
        northeast: LatLng;
        southwest: LatLng;
      };
    };
    place_id: string;
    types: string[];
  }>;
  status: string;
};
```

```
export type PlacesSearchResult = {
  results: Array<{
    place_id: string;
    name: string;
    formatted_address: string;
    geometry: {
      location: LatLng;
    };
  }>;
```

```
    rating?: number;
    user_ratings_total?: number;
    business_status?: string;
    types: string[];
  }>;
  status: string;
};

export type PlaceDetailsResult = {
  result: {
    place_id: string;
    name: string;
    formatted_address: string;
    formatted_phone_number?: string;
    international_phone_number?: string;
    website?: string;
    rating?: number;
    user_ratings_total?: number;
    reviews?: Array<{
      author_name: string;
      rating: number;
      text: string;
      time: number;
    }>;
    opening_hours?: {
      open_now: boolean;
      weekday_text: string[];
    };
    geometry: {
      location: LatLng;
    };
  };
  status: string;
};

export type ElevationResult = {
  results: Array<{
    elevation: number;
    location: LatLng;
    resolution: number;
  }>;
  status: string;
};
```

```

export type TimeZoneResult = {
  dstOffset: number;
  rawOffset: number;
  status: string;
  timeZoneId: string;
  timeZoneName: string;
};

export type RoadsResult = {
  snappedPoints: Array<{
    location: LatLng;
    originalIndex?: number;
    placeId: string;
  }>;
};

// =====
// Google Maps API Reference
// =====

/**
 * GEOCODING - Convert between addresses and coordinates
 * Endpoint: /maps/api/geocode/json
 * Input: { address: string } OR { latlng: string } // latlng: "37.42,-122.084"
 * Output: GeocodingResult // results[0].geometry.location, results[0].formatted_address
 */

/**
 * DIRECTIONS - Get navigation routes between locations
 * Endpoint: /maps/api/directions/json
 * Input: { origin: string, destination: string, mode?: TravelMode, waypoints?: string[] }
 * Output: DirectionsResult // routes[0].legs[0].distance, duration, steps
 */

/**
 * DISTANCE MATRIX - Calculate travel times/distances for multiple origin/destination pairs
 * Endpoint: /maps/api/distancematrix/json
 * Input: { origins: string, destinations: string, mode?: TravelMode, units?: Unit }
 * Output: DistanceMatrixResult // rows[0].elements[1] = first origin to first destination
 */

/**
 * PLACE SEARCH - Find businesses/POIs by text query
 * Endpoint: /maps/api/place/textsearch/json

```

```

* Input: { query: string, location?: string, radius?: number, type?: string }
* Output: PlacesSearchResult // results[].name, rating, geometry.location
*/

/**
* NEARBY SEARCH - Find places near a specific location
* Endpoint: /maps/api/place/nearbysearch/json
* Input: { location: string, radius: number, type?: string, keyword?: string }
* Output: PlacesSearchResult
*/

/**
* PLACE DETAILS - Get comprehensive information about a specific place
* Endpoint: /maps/api/place/details/json
* Input: { place_id: string, fields?: string } // fields: "name,rating,opening_hours,etc"
* Output: PlaceDetailsResult // result.name, rating, opening_hours, etc
*/

/**
* ELEVATION - Get altitude data for geographic points
* Endpoint: /maps/api/elevation/json
* Input: { locations?: string, path?: string, samples?: number } // locations: "lat,lng|lat,lng"
* Output: ElevationResult // results[].elevation (meters)
*/

/**
* TIME ZONE - Get timezone information for a location
* Endpoint: /maps/api/timezone/json
* Input: { location: string, timestamp: number } // timestamp: Math.floor(Date.now() / 1000)
* Output: TimeZoneResult // timeZoneId, timeZoneName
*/

/**
* ROADS - Snap GPS traces to roads, find nearest roads, get speed limits
* - /v1/snapToRoads: Input: { path: string, interpolate?: boolean } // path: "lat,lng|lat,lng"
* - /v1/nearestRoads: Input: { points: string } // points: "lat,lng|lat,lng"
* - /v1/speedLimits: Input: { path: string, units?: SpeedUnit }
* Output: RoadsResult
*/

/**
* PLACE AUTOCOMPLETE - Real-time place suggestions as user types
* Endpoint: /maps/api/place/autocomplete/json
* Input: { input: string, location?: string, radius?: number }

```

```
* Output: { predictions: Array<{ description: string, place_id: string }>:  
*/
```

```
/**
```

```
* STATIC MAPS - Generate map images as URLs (for emails, reports, <img> .  
* Endpoint: /maps/api/staticmap  
* Input: URL params - center: string, zoom: number, size: string, markers:  
* Output: Image URL (not JSON) - use directly in <img src={url} />  
* Note: Construct URL manually with getMapsConfig() for auth  
*/
```

File: ./server/_core/notification.ts

```
import { TRPCError } from "@trpc/server";
import { ENV } from "../env";

export type NotificationPayload = {
  title: string;
  content: string;
};

const TITLE_MAX_LENGTH = 1200;
const CONTENT_MAX_LENGTH = 20000;

const trimValue = (value: string): string => value.trim();
const isEmptyString = (value: unknown): value is string =>
  typeof value === "string" && value.trim().length > 0;

const buildEndpointUrl = (baseUrl: string): string => {
  const normalizedBase = baseUrl.endsWith("/")
    ? baseUrl
    : `${baseUrl}/`;
  return new URL(
    "webdevtoken.v1.WebDevService/SendNotification",
    normalizedBase
  ).toString();
};

const validatePayload = (input: NotificationPayload): NotificationPayload {
  if (!isEmptyString(input.title)) {
    throw new TRPCError({
      code: "BAD_REQUEST",
      message: "Notification title is required.",
    });
  }
  if (!isEmptyString(input.content)) {
    throw new TRPCError({
      code: "BAD_REQUEST",
      message: "Notification content is required.",
    });
  }

  const title = trimValue(input.title);
  const content = trimValue(input.content);
```

```

    if (title.length > TITLE_MAX_LENGTH) {
        throw new TRPCError({
            code: "BAD_REQUEST",
            message: `Notification title must be at most ${TITLE_MAX_LENGTH} characters`
        });
    }

    if (content.length > CONTENT_MAX_LENGTH) {
        throw new TRPCError({
            code: "BAD_REQUEST",
            message: `Notification content must be at most ${CONTENT_MAX_LENGTH} characters`
        });
    }

    return { title, content };
};

/**
 * Dispatches a project-owner notification through the Manus Notification Service
 * Returns `true` if the request was accepted, `false` when the upstream service
 * cannot be reached (callers can fall back to email/slack). Validation errors
 * bubble up as TRPC errors so callers can fix the payload.
 */
export async function notifyOwner(
    payload: NotificationPayload
): Promise<boolean> {
    const { title, content } = validatePayload(payload);

    if (!ENV.forgeApiUrl) {
        throw new TRPCError({
            code: "INTERNAL_SERVER_ERROR",
            message: "Notification service URL is not configured.",
        });
    }

    if (!ENV.forgeApiKey) {
        throw new TRPCError({
            code: "INTERNAL_SERVER_ERROR",
            message: "Notification service API key is not configured.",
        });
    }

    const endpoint = buildEndpointUrl(ENV.forgeApiUrl);

```

```

try {
  const response = await fetch(endpoint, {
    method: "POST",
    headers: {
      accept: "application/json",
      authorization: `Bearer ${ENV.forgeApiKey}`,
      "content-type": "application/json",
      "connect-protocol-version": "1",
    },
    body: JSON.stringify({ title, content }),
  });

  if (!response.ok) {
    const detail = await response.text().catch(() => "");
    console.warn(
      `[Notification] Failed to notify owner (${response.status} ${response.statusText} ${detail ? `: ${detail}` : ""})`
    );
    return false;
  }

  return true;
} catch (error) {
  console.warn("[Notification] Error calling notification service:", error);
  return false;
}
}

```

File: ./server/_core/oauth.ts

```
import { COOKIE_NAME, ONE_YEAR_MS } from "@shared/const";
import type { Express, Request, Response } from "express";
import * as db from "../db";
import { getSessionCookieOptions } from "../cookies";
import { sdk } from "../sdk";

function getQueryParam(req: Request, key: string): string | undefined {
  const value = req.query[key];
  return typeof value === "string" ? value : undefined;
}

export function registerOAuthRoutes(app: Express) {
  app.get("/api/oauth/callback", async (req: Request, res: Response) => {
    const code = getQueryParam(req, "code");
    const state = getQueryParam(req, "state");

    if (!code || !state) {
      res.status(400).json({ error: "code and state are required" });
      return;
    }

    try {
      const tokenResponse = await sdk.exchangeCodeForToken(code, state);
      const userInfo = await sdk.getUserInfo(tokenResponse.accessToken);

      if (!userInfo.openId) {
        res.status(400).json({ error: "openId missing from user info" });
        return;
      }

      await db.upsertUser({
        openId: userInfo.openId,
        name: userInfo.name || null,
        email: userInfo.email ?? null,
        loginMethod: userInfo.loginMethod ?? userInfo.platform ?? null,
        lastSignedIn: new Date(),
      });

      const sessionToken = await sdk.createSessionToken(userInfo.openId, {
        name: userInfo.name || "",
        expiresInMs: ONE_YEAR_MS,
      });
    }
  });
}
```

```
});

const cookieOptions = getSessionCookieOptions(req);
res.cookie(COOKIE_NAME, sessionToken, { ...cookieOptions, maxAge: 0

res.redirect(302, "/");
} catch (error) {
  console.error("[OAuth] Callback failed", error);
  res.status(500).json({ error: "OAuth callback failed" });
}
});
}
```

File: ./server/_core/sdk.ts

```
import { AXIOS_TIMEOUT_MS, COOKIE_NAME, ONE_YEAR_MS } from "@shared/constants";
import { ForbiddenError } from "@shared/_core/errors";
import axios, { type AxiosInstance } from "axios";
import { parse as parseCookieHeader } from "cookie";
import type { Request } from "express";
import { SignJWT, jwtVerify } from "jose";
import type { User } from "../../drizzle/schema";
import * as db from "../db";
import { ENV } from "../env";
import type {
  ExchangeTokenRequest,
  ExchangeTokenResponse,
  GetUserInfoResponse,
  GetUserInfoWithJwtRequest,
  GetUserInfoWithJwtResponse,
} from "../types/manusTypes";
// Utility function
const isEmptyString = (value: unknown): value is string =>
  typeof value === "string" && value.length > 0;

export type SessionPayload = {
  openId: string;
  appId: string;
  name: string;
};

const EXCHANGE_TOKEN_PATH = `/webdev.v1.WebDevAuthPublicService/ExchangeToken`;
const GET_USER_INFO_PATH = `/webdev.v1.WebDevAuthPublicService/GetUserInfo`;
const GET_USER_INFO_WITH_JWT_PATH = `/webdev.v1.WebDevAuthPublicService/GetUserInfoWithJwt`;

class OAuthService {
  constructor(private client: ReturnType<typeof axios.create>) {
    console.log("[OAuth] Initialized with baseURL:", ENV.oAuthServerUrl);
    if (!ENV.oAuthServerUrl) {
      console.error(
        "[OAuth] ERROR: OAUTH_SERVER_URL is not configured! Set OAUTH_SERVER_URL"
      );
    }
  }

  private decodeState(state: string): string {
    const url = new URL(state, "http://localhost");
    const searchParams = url.searchParams;
    const stateObj = {
      url: url.pathname + url.search,
      nonce: searchParams.get("nonce"),
      openId: searchParams.get("openId"),
      appId: searchParams.get("appId"),
      name: searchParams.get("name"),
    };
    return JSON.stringify(stateObj);
  }

  private async exchangeToken(
    request: ExchangeTokenRequest
  ): Promise<ExchangeTokenResponse> {
    const { code, redirectUri } = request;
    const { data } = await axios.post<ExchangeTokenResponse>(
      EXCHANGE_TOKEN_PATH,
      { code, redirectUri },
      {
        headers: { "Content-Type": "application/json" },
      }
    );
    return data;
  }

  private async getUserInfo(
    request: GetUserInfoRequest
  ): Promise<GetUserInfoResponse> {
    const { openId } = request;
    const { data } = await axios.get<GetUserInfoResponse>(
      GET_USER_INFO_PATH,
      {
        headers: { "Content-Type": "application/json" },
      }
    );
    return data;
  }

  private async getUserInfoWithJwt(
    request: GetUserInfoWithJwtRequest
  ): Promise<GetUserInfoWithJwtResponse> {
    const { openId, jwt } = request;
    const { data } = await axios.get<GetUserInfoWithJwtResponse>(
      GET_USER_INFO_WITH_JWT_PATH,
      {
        headers: { "Content-Type": "application/json" },
      }
    );
    return data;
  }
}
```

```

    const redirectUri = atob(state);
    return redirectUri;
}

async getTokenByCode(
  code: string,
  state: string
): Promise<ExchangeTokenResponse> {
  const payload: ExchangeTokenRequest = {
    clientId: ENV.appId,
    grantType: "authorization_code",
    code,
    redirectUri: this.decodeState(state),
  };

  const { data } = await this.client.post<ExchangeTokenResponse>(
    EXCHANGE_TOKEN_PATH,
    payload
  );

  return data;
}

async getUserInfoByToken(
  token: ExchangeTokenResponse
): Promise<GetUserInfoResponse> {
  const { data } = await this.client.post<GetUserInfoResponse>(
    GET_USER_INFO_PATH,
    {
      accessToken: token.accessToken,
    }
  );

  return data;
}

const createOAuthHttpClient = (): AxiosInstance =>
  axios.create({
    baseURL: ENV.oAuthServerUrl,
    timeout: AXIOS_TIMEOUT_MS,
  });

class SDKServer {

```

```

private readonly client: AxiosInstance;
private readonly oauthService: OAuthService;

constructor(client: AxiosInstance = createOAuthHttpClient()) {
  this.client = client;
  this.oauthService = new OAuthService(this.client);
}

private deriveLoginMethod(
  platforms: unknown,
  fallback: string | null | undefined
): string | null {
  if (fallback && fallback.length > 0) return fallback;
  if (!Array.isArray(platforms) || platforms.length === 0) return null;
  const set = new Set<string>(
    platforms.filter((p): p is string => typeof p === "string")
  );
  if (set.has("REGISTERED_PLATFORM_EMAIL")) return "email";
  if (set.has("REGISTERED_PLATFORM_GOOGLE")) return "google";
  if (set.has("REGISTERED_PLATFORM_APPLE")) return "apple";
  if (
    set.has("REGISTERED_PLATFORM_MICROSOFT") ||
    set.has("REGISTERED_PLATFORM_AZURE")
  )
    return "microsoft";
  if (set.has("REGISTERED_PLATFORM_GITHUB")) return "github";
  const first = Array.from(set)[0];
  return first ? first.toLowerCase() : null;
}

/**
 * Exchange OAuth authorization code for access token
 * @example
 * const tokenResponse = await sdk.exchangeCodeForToken(code, state);
 */
async exchangeCodeForToken(
  code: string,
  state: string
): Promise<ExchangeTokenResponse> {
  return this.oauthService.getTokenByCode(code, state);
}

/**
 * Get user information using access token

```

```

* @example
* const userInfo = await sdk.getUserInfo(tokenResponse.accessToken);
*/
async getUserInfo(accessToken: string): Promise<GetUserInfoResponse> {
  const data = await this.oauthService.getUserInfoByToken({
    accessToken,
  } as ExchangeTokenResponse);
  const loginMethod = this.deriveLoginMethod(
    (data as any)?.platforms,
    (data as any)?.platform ?? data.platform ?? null
  );
  return {
    ...(data as any),
    platform: loginMethod,
    loginMethod,
  } as GetUserInfoResponse;
}

private parseCookies(cookieHeader: string | undefined) {
  if (!cookieHeader) {
    return new Map<string, string>();
  }

  const parsed = parseCookieHeader(cookieHeader);
  return new Map(Object.entries(parsed));
}

private getSessionSecret() {
  const secret = ENV.cookieSecret;
  return new TextEncoder().encode(secret);
}

/**
 * Create a session token for a Manus user openId
 * @example
 * const sessionToken = await sdk.createSessionToken(userInfo.openId);
 */
async createSessionToken(
  openId: string,
  options: { expiresInMs?: number; name?: string } = {}
): Promise<string> {
  return this.signSession(
    {
      openId,
    }
  );
}

```

```

        appId: ENV.appId,
        name: options.name || "",
    },
    options
);
}

async signSession(
    payload: SessionPayload,
    options: { expiresInMs?: number } = {}
): Promise<string> {
    const issuedAt = Date.now();
    const expiresInMs = options.expiresInMs ?? ONE_YEAR_MS;
    const expirationSeconds = Math.floor((issuedAt + expiresInMs) / 1000)
    const secretKey = this.getSessionSecret();

    return new SignJWT({
        openId: payload.openId,
        appId: payload.appId,
        name: payload.name,
    })
        .setProtectedHeader({ alg: "HS256", typ: "JWT" })
        .setExpirationTime(expirationSeconds)
        .sign(secretKey);
}

async verifySession(
    cookieValue: string | undefined | null
): Promise<{ openId: string; appId: string; name: string } | null> {
    if (!cookieValue) {
        console.warn("[Auth] Missing session cookie");
        return null;
    }

    try {
        const secretKey = this.getSessionSecret();
        const { payload } = await jwtVerify(cookieValue, secretKey, {
            algorithms: ["HS256"],
        });
        const { openId, appId, name } = payload as Record<string, unknown>;

        if (
            !isNonEmptyString(openId) ||
            !isNonEmptyString(appId) ||

```

```

        !isEmptyString(name)
    ) {
        console.warn("[Auth] Session payload missing required fields");
        return null;
    }

    return {
        openId,
        appId,
        name,
    };
} catch (error) {
    console.warn("[Auth] Session verification failed", String(error));
    return null;
}
}

async getUserInfoWithJwt(
    jwtToken: string
): Promise<GetUserInfoWithJwtResponse> {
    const payload: GetUserInfoWithJwtRequest = {
        jwtToken,
        projectId: ENV.appId,
    };

    const { data } = await this.client.post<GetUserInfoWithJwtResponse>(
        GET_USER_INFO_WITH_JWT_PATH,
        payload
    );

    const loginMethod = this.deriveLoginMethod(
        (data as any)?.platforms,
        (data as any)?.platform ?? data.platform ?? null
    );
    return {
        ...(data as any),
        platform: loginMethod,
        loginMethod,
    } as GetUserInfoWithJwtResponse;
}

async authenticateRequest(req: Request): Promise<AuthenticatedUser> {
    // Regular authentication flow
    const cookies = this.parseCookies(req.headers.cookie);

```

```

const sessionCookie = cookies.get(COOKIE_NAME);
const session = await this.verifySession(sessionCookie);

if (!session) {
  throw ForbiddenError("Invalid session cookie");
}

if (session.openId.startsWith(CRON_OPEN_ID_PREFIX)) {
  const userInfo = await this.getUserInfoWithJwt(sessionCookie ?? "")
  const taskUid = userInfo.taskUid ?? null;
  if (!taskUid) {
    throw ForbiddenError("Cron session missing task_uid");
  }
  return buildCronUser(userInfo);
}

const sessionUserId = session.openId;
const signedInAt = new Date();
let user = await db.getUserByOpenId(sessionUserId);

// If user not in DB, sync from OAuth server automatically
if (!user) {
  try {
    const userInfo = await this.getUserInfoWithJwt(sessionCookie ?? "")
    await db.upsertUser({
      openId: userInfo.openId,
      name: userInfo.name || null,
      email: userInfo.email ?? null,
      loginMethod: userInfo.loginMethod ?? userInfo.platform ?? null,
      lastSignedIn: signedInAt,
    });
    user = await db.getUserByOpenId(userInfo.openId);
  } catch (error) {
    console.error("[Auth] Failed to sync user from OAuth:", error);
    throw ForbiddenError("Failed to sync user info");
  }
}

if (!user) {
  throw ForbiddenError("User not found");
}

await db.upsertUser({
  openId: user.openId,

```

```

        lastSignedIn: signedInAt,
    });

    return user;
}
}

const CRON_OPEN_ID_PREFIX = "cron_";

/** Result of `sdk.authenticateRequest`. Cron callbacks set `isCron=true`
export type AuthenticatedUser = User & {
    taskUid?: string;
    isCron?: boolean;
};

function buildCronUser(
    userInfo: GetUserInfoWithJwtResponse
): AuthenticatedUser {
    const now = new Date();
    return {
        id: -1,
        openId: userInfo.openId,
        name: userInfo.name || "Manus Scheduled Task",
        email: null,
        loginMethod: null,
        role: "user",
        createdAt: now,
        updatedAt: now,
        lastSignedIn: now,
        taskUid: userInfo.taskUid ?? undefined,
        isCron: true,
    } as AuthenticatedUser;
}

export const sdk = new SDKServer();

```

File: ./server/_core/storageProxy.ts

```
import type { Express } from "express";
import { ENV } from "../env";

export function registerStorageProxy(app: Express) {
  app.get("/manus-storage/*", async (req, res) => {
    const key = (req.params as Record<string, string>)[0];
    if (!key) {
      res.status(400).send("Missing storage key");
      return;
    }

    if (!ENV.forgeApiUrl || !ENV.forgeApiKey) {
      res.status(500).send("Storage proxy not configured");
      return;
    }

    try {
      const forgeUrl = new URL(
        "v1/storage/presign/get",
        ENV.forgeApiUrl.replace(/\/+$/, "") + "/",
      );
      forgeUrl.searchParams.set("path", key);

      const forgeResp = await fetch(forgeUrl, {
        headers: { Authorization: `Bearer ${ENV.forgeApiKey}` },
      });

      if (!forgeResp.ok) {
        const body = await forgeResp.text().catch(() => "");
        console.error(`[StorageProxy] forge error: ${forgeResp.status} ${body}`);
        res.status(502).send("Storage backend error");
        return;
      }

      const { url } = (await forgeResp.json()) as { url: string };
      if (!url) {
        res.status(502).send("Empty signed URL from backend");
        return;
      }

      res.set("Cache-Control", "no-store");
    }
  });
}
```

```

    res.redirect(307, url);
  } catch (err) {
    console.error("[StorageProxy] failed:", err);
    res.status(502).send("Storage proxy error");
  }
});
}

```

File: ./server/_core/systemRouter.ts

```

import { z } from "zod";
import { notifyOwner } from "../notification";
import { adminProcedure, publicProcedure, router } from "../trpc";

export const systemRouter = router({
  health: publicProcedure
    .input(
      z.object({
        timestamp: z.number().min(0, "timestamp cannot be negative"),
      })
    )
    .query(() => ({
      ok: true,
    })),

  notifyOwner: adminProcedure
    .input(
      z.object({
        title: z.string().min(1, "title is required"),
        content: z.string().min(1, "content is required"),
      })
    )
    .mutation(async ({ input }) => {
      const delivered = await notifyOwner(input);
      return {
        success: delivered,
      } as const;
    }),
});

```

File: ./server/_core/trpc.ts

```
import { NOT_ADMIN_ERR_MSG, UNAUTHED_ERR_MSG } from '@shared/const';
import { initTRPC, TRPCError } from '@trpc/server';
import superjson from 'superjson';
import type { TrpcContext } from '../context';

const t = initTRPC.context<TrpcContext>().create({
  transformer: superjson,
});

export const router = t.router;
export const publicProcedure = t.procedure;

const requireUser = t.middleware(async opts => {
  const { ctx, next } = opts;

  if (!ctx.user) {
    throw new TRPCError({ code: 'UNAUTHORIZED', message: UNAUTHED_ERR_MSG });
  }

  return next({
    ctx: {
      ...ctx,
      user: ctx.user,
    },
  });
});

export const protectedProcedure = t.procedure.use(requireUser);

export const adminProcedure = t.procedure.use(
  t.middleware(async opts => {
    const { ctx, next } = opts;

    if (!ctx.user || ctx.user.role !== 'admin') {
      throw new TRPCError({ code: 'FORBIDDEN', message: NOT_ADMIN_ERR_MSG });
    }

    return next({
      ctx: {
        ...ctx,
        user: ctx.user,
      },
    });
  })
);
```

```
    },  
  });  
},  
);
```

File: ./server/_core/types/cookie.d.ts

```
declare module "cookie" {  
  export function parse(  
    str: string,  
    options?: Record<string, unknown>  
  ): Record<string, string>;  
}
```

File: ./server/_core/types/manusTypes.ts

```
// WebDev Auth TypeScript types
// Auto-generated from protobuf definitions
// Generated on: 2025-09-24T05:57:57.338Z

export interface AuthorizeRequest {
  redirectUri: string;
  projectId: string;
  state: string;
  responseType: string;
  scope: string;
}

export interface AuthorizeResponse {
  redirectUrl: string;
}

export interface ExchangeTokenRequest {
  grantType: string;
  code: string;
  refreshToken?: string;
  clientId: string;
  clientSecret?: string;
  redirectUri: string;
}

export interface ExchangeTokenResponse {
  accessToken: string;
  tokenType: string;
  expiresIn: number;
  refreshToken?: string;
  scope: string;
  idToken: string;
}

export interface GetUserInfoRequest {
  accessToken: string;
}

export interface GetUserInfoResponse {
  openId: string;
  projectId: string;
}
```

```
    name: string;
    email?: string | null;
    platform?: string | null;
    loginMethod?: string | null;
}

export interface CanAccessRequest {
    openId: string;
    projectId: string;
}

export interface CanAccessResponse {
    canAccess: boolean;
}

export interface GetUserInfoWithJwtRequest {
    jwtToken: string;
    projectId: string;
}

export interface GetUserInfoWithJwtResponse {
    openId: string;
    projectId: string;
    name: string;
    email?: string | null;
    platform?: string | null;
    loginMethod?: string | null;
    /** Cron-only; references `schedule_task.uid`. */
    taskUid?: string | null;
}
```

File: ./server/_core/vite.ts

```
import express, { type Express } from "express";
import fs from "fs";
import { type Server } from "http";
import { nanoid } from "nanoid";
import path from "path";
import { createServer as createViteServer } from "vite";
import viteConfig from "../../vite.config";

export async function setupVite(app: Express, server: Server) {
  const serverOptions = {
    middlewareMode: true,
    hmr: { server },
    allowedHosts: true as const,
  };

  const vite = await createViteServer({
    ...viteConfig,
    configFile: false,
    server: serverOptions,
    appType: "custom",
  });

  app.use(vite.middlewares);
  app.use("*", async (req, res, next) => {
    const url = req.originalUrl;

    try {
      const clientTemplate = path.resolve(
        import.meta.dirname,
        "../..",
        "client",
        "index.html"
      );

      // always reload the index.html file from disk incase it changes
      let template = await fs.promises.readFile(clientTemplate, "utf-8");
      template = template.replace(
        `src="/src/main.tsx"`,
        `src="/src/main.tsx?v=${nanoid()}"`
      );

      const page = await vite.transformIndexHtml(url, template);
```

```

    res.status(200).set({ "Content-Type": "text/html" }).end(page);
  } catch (e) {
    vite.ssrFixStacktrace(e as Error);
    next(e);
  }
});
}

export function serveStatic(app: Express) {
  const distPath =
    process.env.NODE_ENV === "development"
      ? path.resolve(import.meta.dirname, "../..", "dist", "public")
      : path.resolve(import.meta.dirname, "public");
  if (!fs.existsSync(distPath)) {
    console.error(
      `Could not find the build directory: ${distPath}, make sure to build
    );
  }

  app.use(express.static(distPath));

  // fall through to index.html if the file doesn't exist
  app.use("*", (_req, res) => {
    res.sendFile(path.resolve(distPath, "index.html"));
  });
}

```

File: ./server/_core/voiceTranscription.ts

```
/**
 * Voice transcription helper using internal Speech-to-Text service
 *
 * Frontend implementation guide:
 * 1. Capture audio using MediaRecorder API
 * 2. Upload audio to storage (e.g., S3) to get URL
 * 3. Call transcription with the URL
 *
 * Example usage:
 * ```tsx
 * // Frontend component
 * const transcribeMutation = trpc.voice.transcribe.useMutation({
 *   onSuccess: (data) => {
 *     console.log(data.text); // Full transcription
 *     console.log(data.language); // Detected language
 *     console.log(data.segments); // Timestamped segments
 *   }
 * });
 *
 * // After uploading audio to storage
 * transcribeMutation.mutate({
 *   audioUrl: uploadedAudioUrl,
 *   language: 'en', // optional
 *   prompt: 'Transcribe the meeting' // optional
 * });
 * ```
 */
import { ENV } from './env';

export type TranscribeOptions = {
  audioUrl: string; // URL to the audio file (e.g., S3 URL)
  language?: string; // Optional: specify language code (e.g., "en", "es")
  prompt?: string; // Optional: custom prompt for the transcription
};

// Native Whisper API segment format
export type WhisperSegment = {
  id: number;
  seek: number;
  start: number;
  end: number;
};
```

```

    text: string;
    tokens: number[];
    temperature: number;
    avg_logprob: number;
    compression_ratio: number;
    no_speech_prob: number;
};

// Native Whisper API response format
export type WhisperResponse = {
    task: "transcribe";
    language: string;
    duration: number;
    text: string;
    segments: WhisperSegment[];
};

export type TranscriptionResponse = WhisperResponse; // Return native Whisper response

export type TranscriptionError = {
    error: string;
    code: "FILE_TOO_LARGE" | "INVALID_FORMAT" | "TRANSCRIPTION_FAILED" | "UNRECOGNIZED_SPEECH";
    details?: string;
};

/**
 * Transcribe audio to text using the internal Speech-to-Text service
 *
 * @param options - Audio data and metadata
 * @returns Transcription result or error
 */
export async function transcribeAudio(
    options: TranscribeOptions
): Promise<TranscriptionResponse | TranscriptionError> {
    try {
        // Step 1: Validate environment configuration
        if (!ENV.forgeApiUrl) {
            return {
                error: "Voice transcription service is not configured",
                code: "SERVICE_ERROR",
                details: "BUILT_IN_FORGE_API_URL is not set"
            };
        }
        if (!ENV.forgeApiKey) {

```

```

    return {
      error: "Voice transcription service authentication is missing",
      code: "SERVICE_ERROR",
      details: "BUILT_IN_FORGE_API_KEY is not set"
    };
  }

  // Step 2: Download audio from URL
  let audioBuffer: Buffer;
  let mimeType: string;
  try {
    const response = await fetch(options.audioUrl);
    if (!response.ok) {
      return {
        error: "Failed to download audio file",
        code: "INVALID_FORMAT",
        details: `HTTP ${response.status}: ${response.statusText}`
      };
    }

    audioBuffer = Buffer.from(await response.arrayBuffer());
    mimeType = response.headers.get('content-type') || 'audio/mpeg';

    // Check file size (16MB limit)
    const sizeMB = audioBuffer.length / (1024 * 1024);
    if (sizeMB > 16) {
      return {
        error: "Audio file exceeds maximum size limit",
        code: "FILE_TOO_LARGE",
        details: `File size is ${sizeMB.toFixed(2)}MB, maximum allowed :
      };
    }
  } catch (error) {
    return {
      error: "Failed to fetch audio file",
      code: "SERVICE_ERROR",
      details: error instanceof Error ? error.message : "Unknown error"
    };
  }

  // Step 3: Create FormData for multipart upload to Whisper API
  const formData = new FormData();

  // Create a Blob from the buffer and append to form

```

```

const filename = `audio.${getFileExtension(mimeType)}`;
const audioBlob = new Blob([new Uint8Array(audioBuffer)], { type: mimeType });
formData.append("file", audioBlob, filename);

formData.append("model", "whisper-1");
formData.append("response_format", "verbose_json");

// Add prompt - use custom prompt if provided, otherwise generate based on language
const prompt = options.prompt || (
  options.language
    ? `Transcribe the user's voice to text, the user's working language is ${options.language}`
    : "Transcribe the user's voice to text"
);
formData.append("prompt", prompt);

// Step 4: Call the transcription service
const baseUrl = ENV.forgeApiUrl.endsWith("/")
  ? ENV.forgeApiUrl
  : `${ENV.forgeApiUrl}/`;

const fullUrl = new URL(
  "v1/audio/transcriptions",
  baseUrl
).toString();

const response = await fetch(fullUrl, {
  method: "POST",
  headers: {
    authorization: `Bearer ${ENV.forgeApiKey}`,
    "Accept-Encoding": "identity",
  },
  body: formData,
});

if (!response.ok) {
  const errorText = await response.text().catch(() => "");
  return {
    error: "Transcription service request failed",
    code: "TRANSCRIPTION_FAILED",
    details: `${response.status} ${response.statusText}${errorText} ?
  };
}

// Step 5: Parse and return the transcription result

```

```

const whisperResponse = await response.json() as WhisperResponse;

// Validate response structure
if (!whisperResponse.text || typeof whisperResponse.text !== 'string') {
    return {
        error: "Invalid transcription response",
        code: "SERVICE_ERROR",
        details: "Transcription service returned an invalid response format"
    };
}

return whisperResponse; // Return native Whisper API response directly

} catch (error) {
    // Handle unexpected errors
    return {
        error: "Voice transcription failed",
        code: "SERVICE_ERROR",
        details: error instanceof Error ? error.message : "An unexpected error occurred"
    };
}
}

/**
 * Helper function to get file extension from MIME type
 */
function getFileExtension(mimeType: string): string {
    const mimeToExt: Record<string, string> = {
        'audio/webm': 'webm',
        'audio/mp3': 'mp3',
        'audio/mpeg': 'mp3',
        'audio/wav': 'wav',
        'audio/wave': 'wav',
        'audio/ogg': 'ogg',
        'audio/m4a': 'm4a',
        'audio/mp4': 'm4a',
    };

    return mimeToExt[mimeType] || 'audio';
}

/**
 * Helper function to get full language name from ISO code
 */

```

```

function getLanguageName(langCode: string): string {
  const langMap: Record<string, string> = {
    'en': 'English',
    'es': 'Spanish',
    'fr': 'French',
    'de': 'German',
    'it': 'Italian',
    'pt': 'Portuguese',
    'ru': 'Russian',
    'ja': 'Japanese',
    'ko': 'Korean',
    'zh': 'Chinese',
    'ar': 'Arabic',
    'hi': 'Hindi',
    'nl': 'Dutch',
    'pl': 'Polish',
    'tr': 'Turkish',
    'sv': 'Swedish',
    'da': 'Danish',
    'no': 'Norwegian',
    'fi': 'Finnish',
  };

  return langMap[langCode] || langCode;
}

/**
 * Example tRPC procedure implementation:
 *
 * ```ts
 * // In server/routers.ts
 * import { transcribeAudio } from "../_core/voiceTranscription";
 *
 * export const voiceRouter = router({
 *   transcribe: protectedProcedure
 *     .input(z.object({
 *       audioUrl: z.string(),
 *       language: z.string().optional(),
 *       prompt: z.string().optional(),
 *     }))
 *     .mutation(async ({ input, ctx }) => {
 *       const result = await transcribeAudio(input);
 *
 *       // Check if it's an error

```

```

*         if ('error' in result) {
*             throw new TRPCError({
*                 code: 'BAD_REQUEST',
*                 message: result.error,
*                 cause: result,
*             });
*         }
*
*         // Optionally save transcription to database
*         await db.insert(transcriptions).values({
*             userId: ctx.user.id,
*             text: result.text,
*             duration: result.duration,
*             language: result.language,
*             audioUrl: input.audioUrl,
*             createdAt: new Date(),
*         });
*
*         return result;
*     }},
* });
*
*/

```

File: ./server/auth.logout.test.ts

```
import { describe, expect, it } from "vitest";
import { appRouter } from "../routers";
import { COOKIE_NAME } from "../../shared/const";
import type { TrpcContext } from "../_core/context";

type CookieCall = {
  name: string;
  options: Record<string, unknown>;
};

type AuthenticatedUser = NonNullable<TrpcContext["user"]>;

function createAuthContext(): { ctx: TrpcContext; clearedCookies: CookieCall[] } {
  const clearedCookies: CookieCall[] = [];

  const user: AuthenticatedUser = {
    id: 1,
    openId: "sample-user",
    email: "sample@example.com",
    name: "Sample User",
    loginMethod: "manus",
    role: "user",
    createdAt: new Date(),
    updatedAt: new Date(),
    lastSignedIn: new Date(),
  };

  const ctx: TrpcContext = {
    user,
    req: {
      protocol: "https",
      headers: {},
    } as TrpcContext["req"],
    res: {
      clearCookie: (name: string, options: Record<string, unknown>) => {
        clearedCookies.push({ name, options });
      },
    } as TrpcContext["res"],
  };

  return { ctx, clearedCookies };
}
```

```
}

describe("auth.logout", () => {
  it("clears the session cookie and reports success", async () => {
    const { ctx, clearedCookies } = createAuthContext();
    const caller = appRouter.createCaller(ctx);

    const result = await caller.auth.logout();

    expect(result).toEqual({ success: true });
    expect(clearedCookies).toHaveLength(1);
    expect(clearedCookies[0]?.name).toBe(COOKIE_NAME);
    expect(clearedCookies[0]?.options).toMatchObject({
      maxAge: -1,
      secure: true,
      sameSite: "none",
      httpOnly: true,
      path: "/",
    });
  });
});
```

File: ./server/db.ts

```
import { eq } from "drizzle-orm";
import { drizzle } from "drizzle-orm/mysql2";
import { InsertUser, users } from "../drizzle/schema";
import { ENV } from '../_core/env';

let _db: ReturnType<typeof drizzle> | null = null;

// Lazily create the drizzle instance so local tooling can run without a db
export async function getDb() {
  if (!_db && process.env.DATABASE_URL) {
    try {
      _db = drizzle(process.env.DATABASE_URL);
    } catch (error) {
      console.warn("[Database] Failed to connect:", error);
      _db = null;
    }
  }
  return _db;
}

export async function upsertUser(user: InsertUser): Promise<void> {
  if (!user.openId) {
    throw new Error("User openId is required for upsert");
  }

  const db = await getDb();
  if (!db) {
    console.warn("[Database] Cannot upsert user: database not available");
    return;
  }

  try {
    const values: InsertUser = {
      openId: user.openId,
    };
    const updateSet: Record<string, unknown> = {};

    const textFields = ["name", "email", "loginMethod"] as const;
    type TextField = (typeof textFields)[number];

    const assignNullable = (field: TextField) => {
```

```

    const value = user[field];
    if (value === undefined) return;
    const normalized = value ?? null;
    values[field] = normalized;
    updateSet[field] = normalized;
  };

  textFields.forEach(assignNullable);

  if (user.lastSignedIn !== undefined) {
    values.lastSignedIn = user.lastSignedIn;
    updateSet.lastSignedIn = user.lastSignedIn;
  }
  if (user.role !== undefined) {
    values.role = user.role;
    updateSet.role = user.role;
  } else if (user.openId === ENV.ownerOpenId) {
    values.role = 'admin';
    updateSet.role = 'admin';
  }

  if (!values.lastSignedIn) {
    values.lastSignedIn = new Date();
  }

  if (Object.keys(updateSet).length === 0) {
    updateSet.lastSignedIn = new Date();
  }

  await db.insert(users).values(values).onDuplicateKeyUpdate({
    set: updateSet,
  });
} catch (error) {
  console.error("[Database] Failed to upsert user:", error);
  throw error;
}

export async function getUserByOpenId(openId: string) {
  const db = await getDb();
  if (!db) {
    console.warn("[Database] Cannot get user: database not available");
    return undefined;
  }
}

```

```
const result = await db.select().from(users).where(eq(users.openId, openId));

return result.length > 0 ? result[0] : undefined;
}

// TODO: add feature queries here as your schema grows.
```

File: ./server/email.ts

```
import nodemailer from "nodemailer";
import { ENV } from "../_core/env";

let transporter: ReturnType<typeof nodemailer.createTransport> | null = null;

function getTransporter() {
  if (!transporter) {
    transporter = nodemailer.createTransport({
      service: "gmail",
      auth: {
        user: ENV.gmailUser,
        pass: ENV.gmailAppPassword,
      },
    });
  }
  return transporter;
}

export async function sendEmail(options: {
  to: string;
  subject: string;
  html: string;
  text?: string;
}) {
  try {
    const transporter = getTransporter();
    const result = await transporter.sendMail({
      from: ENV.gmailUser,
      to: options.to,
      subject: options.subject,
      text: options.text || options.html,
      html: options.html,
    });
    return { success: true, messageId: result.messageId };
  } catch (error) {
    console.error("Error sending email:", error);
    throw error;
  }
}

export async function sendContactFormEmail(data: {
```

```

    name: string;
    email: string;
    message: string;
  }) {
    const html = `
      <div style="font-family: Arial, sans-serif; max-width: 600px; margin:
        <h2 style="color: #00d9ff;">New Portfolio Contact Form Submission</h2>
        <hr style="border: none; border-top: 1px solid #e0e0e0; margin: 20px 0 20px 0;">

        <div style="margin: 20px 0;">
          <p><strong>Name:</strong> ${escapeHtml(data.name)}</p>
          <p><strong>Email:</strong> ${escapeHtml(data.email)}</p>
        </div>

        <div style="margin: 20px 0; background-color: #f5f5f5; padding: 15px 0 15px 20px;">
          <p><strong>Message:</strong></p>
          <p style="white-space: pre-wrap; word-wrap: break-word;">${escapeHtml(data.message)}</p>
        </div>

        <hr style="border: none; border-top: 1px solid #e0e0e0; margin: 20px 0 20px 0;">
        <p style="color: #999; font-size: 12px;">This email was sent from your portfolio</p>
      </div>
    `;

    return sendEmail({
      to: ENV.gmailUser,
      subject: `New Contact: ${data.name}`,
      html,
      text: `Name: ${data.name}\nEmail: ${data.email}\n\nMessage:\n${data.message}`
    });
  }

function escapeHtml(text: string): string {
  const map: Record<string, string> = {
    "&": "&amp;",
    "<": "&lt;",
    ">": "&gt;",
    "'": "&quot;",
    '"': "&#039;",
  };
  return text.replace(/[&<>"']/g, (m) => map[m]);
}

```

File: ./server/index.ts

```
import express from "express";
import { createServer } from "http";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

async function startServer() {
  const app = express();
  const server = createServer(app);

  // Serve static files from dist/public in production
  const staticPath =
    process.env.NODE_ENV === "production"
      ? path.resolve(__dirname, "public")
      : path.resolve(__dirname, "..", "dist", "public");

  app.use(express.static(staticPath));

  // Handle client-side routing - serve index.html for all routes
  app.get("*", (_req, res) => {
    res.sendFile(path.join(staticPath, "index.html"));
  });

  const port = process.env.PORT || 3000;

  server.listen(port, () => {
    console.log(`Server running on http://localhost:${port}/`);
  });
}

startServer().catch(console.error);
```

File: ./server/routers.ts

```
import { COOKIE_NAME } from "@shared/const";
import { getSessionCookieOptions } from "../_core/cookies";
import { systemRouter } from "../_core/systemRouter";
import { publicProcedure, router } from "../_core/trpc";
import { z } from "zod";
import { sendContactFormEmail } from "../email";

export const appRouter = router({
  system: systemRouter,
  auth: router({
    me: publicProcedure.query((opts) => opts.ctx.user),
    logout: publicProcedure.mutation(({ ctx }) => {
      const cookieOptions = getSessionCookieOptions(ctx.req);
      ctx.res.clearCookie(COOKIE_NAME, { ...cookieOptions, maxAge: -1 });
      return {
        success: true,
      } as const;
    }),
  }),
  contact: router({
    sendMessage: publicProcedure
      .input(
        z.object({
          name: z.string().min(1, "Name is required"),
          email: z.string().email("Valid email is required"),
          message: z.string().min(1, "Message is required"),
        })
      )
      .mutation(async ({ input }) => {
        try {
          await sendContactFormEmail({
            name: input.name,
            email: input.email,
            message: input.message,
          });

          return {
            success: true,
            message: "Message sent successfully! I'll get back to you soon",
          };
        } catch {
          // ...
        }
      })
  })
});
```

```
    } catch (error) {  
      console.error("Error sending contact message:", error);  
      return {  
        success: false,  
        message: "Failed to send message. Please try again.",  
      };  
    }  
  }  
},  
});  
});  
});  
  
export type AppRouter = typeof appRouter;
```

File: ./server/storage.ts

```
// Preconfigured storage helpers for Manus WebDev templates
// Uploads via Forge Server presigned URL to S3 (PUT direct).
// Downloads return /manus-storage/{key} paths served via 307 redirect.

import { ENV } from "../_core/env";

function getForgeConfig() {
  const forgeUrl = ENV.forgeApiUrl;
  const forgeKey = ENV.forgeApiKey;

  if (!forgeUrl || !forgeKey) {
    throw new Error(
      "Storage config missing: set BUILT_IN_FORGE_API_URL and BUILT_IN_FORGE_API_KEY"
    );
  }

  return { forgeUrl: forgeUrl.replace(/\/+$/, ""), forgeKey };
}

function normalizeKey(relKey: string): string {
  return relKey.replace(/^\/+/, "");
}

function appendHashSuffix(relKey: string): string {
  const hash = crypto.randomUUID().replace(/-/g, "").slice(0, 8);
  const lastDot = relKey.lastIndexOf(".");
  if (lastDot === -1) return `${relKey}_${hash}`;
  return `${relKey.slice(0, lastDot)}_${hash}${relKey.slice(lastDot)}`;
}

export async function storagePut(
  relKey: string,
  data: Buffer | Uint8Array | string,
  contentType = "application/octet-stream",
): Promise<{ key: string; url: string }> {
  const { forgeUrl, forgeKey } = getForgeConfig();
  const key = appendHashSuffix(normalizeKey(relKey));

  // 1. Get presigned PUT URL from Forge
  const presignUrl = new URL("v1/storage/presign/put", forgeUrl + "/");
  presignUrl.searchParams.set("path", key);
```

```

const presignResp = await fetch(presignUrl, {
  headers: { Authorization: `Bearer ${forgeKey}` },
});

if (!presignResp.ok) {
  const msg = await presignResp.text().catch(() => presignResp.statusText);
  throw new Error(`Storage presign failed (${presignResp.status}): ${msg}`);
}

const { url: s3Url } = (await presignResp.json()) as { url: string };
if (!s3Url) throw new Error("Forge returned empty presign URL");

// 2. PUT file directly to S3
const blob =
  typeof data === "string"
    ? new Blob([data], { type: contentType })
    : new Blob([data as any], { type: contentType });

const uploadResp = await fetch(s3Url, {
  method: "PUT",
  headers: { "Content-Type": contentType },
  body: blob,
});

if (!uploadResp.ok) {
  throw new Error(`Storage upload to S3 failed (${uploadResp.status})`);
}

return { key, url: `/manus-storage/${key}` };
}

export async function storageGet(relKey: string): Promise<{ key: string; url: string }> {
  const key = normalizeKey(relKey);
  return { key, url: `/manus-storage/${key}` };
}

export async function storageGetSignedUrl(relKey: string): Promise<string> {
  const { forgeUrl, forgeKey } = getForgeConfig();
  const key = normalizeKey(relKey);

  const getUrl = new URL("v1/storage/presign/get", forgeUrl + "/");
  getUrl.searchParams.set("path", key);

```

```

const resp = await fetch(getUrl, {
  headers: { Authorization: `Bearer ${forgeKey}` },
});

if (!resp.ok) {
  const msg = await resp.text().catch(() => resp.statusText);
  throw new Error(`Storage signed URL failed (${resp.status}): ${msg}`)
}

const { url } = (await resp.json()) as { url: string };
return url;
}

```

File: ./shared/_core/errors.ts

```

/**
 * Base HTTP error class with status code.
 * Throw this from route handlers to send specific HTTP errors.
 */
export class HttpError extends Error {
  constructor(
    public statusCode: number,
    message: string
  ) {
    super(message);
    this.name = "HttpError";
  }
}

// Convenience constructors
export const BadRequestError = (msg: string) => new HttpError(400, msg);
export const UnauthorizedError = (msg: string) => new HttpError(401, msg);
export const ForbiddenError = (msg: string) => new HttpError(403, msg);
export const NotFoundError = (msg: string) => new HttpError(404, msg);

```

File: ./shared/const.ts

```
export const COOKIE_NAME = "app_session_id";
export const ONE_YEAR_MS = 1000 * 60 * 60 * 24 * 365;
export const AXIOS_TIMEOUT_MS = 30_000;
export const UNAUTHED_ERR_MSG = 'Please login (10001)';
export const NOT_ADMIN_ERR_MSG = 'You do not have required permission (10002)';
```

File: ./shared/types.ts

```
/**
 * Unified type exports
 * Import shared types from this single entry point.
 */

export type * from "../drizzle/schema";
export * from "../_core/errors";
```

File: ./template.json

```
{
  "id": "web-static",
  "name": "Web App (static only)",
  "description": "A pure frontend static website with React and Vite, no backend",
  "capabilities": [
    "static"
  ],
  "files": {
    "package.json": "{\n  \"name\": \"prashanth-portfolio\", \n  \"version\": \"0.0.0\", \n  \"private\": true, \n  \"dependencies\": {\n    \"react\": \"^18.2.0\", \n    \"react-dom\": \"^18.2.0\", \n    \"@fortawesome/fontawesome-svg-core\": \"^6.4.0\", \n    \"@fortawesome/free-solid-svg-icons\": \"^6.4.0\", \n    \"@fortawesome/react-fontawesome\": \"^0.2.0\", \n    \"sonner\": \"^1.4.0\", \n    \"tailwindcss\": \"^3.4.1\" \n  }, \n  \"devDependencies\": {\n    \"vite\": \"^5.0.0\", \n    \"typescript\": \"^5.0.0\", \n    \"eslint\": \"^8.57.0\", \n    \"eslint-plugin-react\": \"^7.33.0\", \n    \"eslint-plugin-react-hooks\": \"^4.6.0\", \n    \"eslint-plugin-sonner\": \"^0.2.0\", \n    \"postcss\": \"^8.4.39\", \n    \"autoprefixer\": \"^10.4.19\" \n  } \n}",
    "client/src/App.tsx": "import { Toaster } from \"@/components/ui/sonner\"; \n\nimport Home from \"./pages/Home\"; \n\nfunction App() { \n  return <Home />; \n} \n\nexport default App;",
    "client/src/pages/Home.tsx": "import { Button } from \"@/components/ui/button\"; \n\nconst Home = () => { \n  return ( \n    <div> \n      <p>Prashanth's Portfolio</p> \n      <p>Full Stack Developer</p> \n      <p>Contact me at prashanthk@gmail.com</p> \n      <p><a href='\"mailto:prashanthk@gmail.com\"'>Email</a></p> \n      <p><a href='\"https://www.linkedin.com/in/prashanthk\"'>LinkedIn</a></p> \n      <p><a href='\"https://github.com/prashanthk\"'>GitHub</a></p> \n    </div>) \n}; \n\nexport default Home;",
    "client/src/index.css": "@import \"tailwindcss\"; \n@import \"tw-animate-css\";",
    "client/index.html": "<!doctype html>\n<html lang=\"en\">\n\n  <head>\n    <meta charset=\"UTF-8\" /\n    <title>Prashanth's Portfolio</title>\n    <script src=\"/js/main.js\" defer></script>\n  </head>\n\n  <body>\n    <div id=\"root\"></div>\n  </body>\n}</html>",
    "server/index.ts": "import express from \"express\"; \nimport { createServer } from \"http\"; \nimport path from \"path\"; \n\nconst app = express(); \n\napp.use(express.static(path.resolve(__dirname, \"../client\"))); \n\nconst server = createServer(app); \n\nserver.listen(3000, () => { \n  console.log(\"Server is running on port 3000\"); \n});"
```

File: ./tsconfig.json

```
{
  "include": ["client/src/**/*", "shared/**/*", "server/**/*"],
  "exclude": ["node_modules", "build", "dist", "**/*.test.ts"],
  "compilerOptions": {
    "incremental": true,
    "tsBuildInfoFile": "./node_modules/typescript/tsbuildinfo",
    "noEmit": true,
    "module": "ESNext",
    "strict": true,
    "lib": ["esnext", "dom", "dom.iterable"],
    "jsx": "preserve",
    "esModuleInterop": true,
    "skipLibCheck": true,
    "allowImportingTsExtensions": true,
    "moduleResolution": "bundler",
    "baseUrl": ".",
    "types": ["node", "vite/client"],
    "paths": {
      "@/*": [".client/src/*"],
      "@shared/*": [".shared/*"]
    }
  }
}
```

File: ./tsconfig.node.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2023"],
    "module": "ESNext",
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "isolatedModules": true,
    "moduleDetection": "force",
    "noEmit": true,

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["vite.config.ts"]
}
```

File: ./vercel.json

```
{
  "version": 2,
  "buildCommand": "pnpm install && pnpm run build",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

File: ./vite.config.ts

```
import { jsxLocPlugin } from "@builder.io/vite-plugin-jsx-loc";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import path from "node:path";
import { defineConfig } from "vite";

export default defineConfig({
  plugins: [react(), tailwindcss(), jsxLocPlugin()],
  resolve: {
    alias: {
      "@": path.resolve(import.meta.dirname, "client", "src"),
      "@shared": path.resolve(import.meta.dirname, "shared"),
      "@assets": path.resolve(import.meta.dirname, "attached_assets"),
    },
  },
  envDir: path.resolve(import.meta.dirname),
  root: path.resolve(import.meta.dirname, "client"),
  publicDir: path.resolve(import.meta.dirname, "client", "public"),
  build: {
    outDir: path.resolve(import.meta.dirname, "dist"),
    emptyOutDir: true,
  },
  server: {
    host: true,
    allowedHosts: ["localhost", "127.0.0.1"],
    fs: {
      strict: true,
      deny: ["**/*. *"],
    },
  },
});
```

File: ./vitest.config.ts

```
import { defineConfig } from "vitest/config";
import path from "path";

const templateRoot = path.resolve(import.meta.dirname);

export default defineConfig({
  root: templateRoot,
  resolve: {
    alias: {
      "@": path.resolve(templateRoot, "client", "src"),
      "@shared": path.resolve(templateRoot, "shared"),
      "@assets": path.resolve(templateRoot, "attached_assets"),
    },
  },
  test: {
    environment: "node",
    include: ["server/**/*.test.ts", "server/**/*.spec.ts"],
  },
});
```